

Zarafshan River Basin Water System Model

A Simulation-Based Coursebook

Dr. Tobias Siegfried

Nicolas Lazaro

Dr. Beatrice Marti

2026-10-06

Table of contents

1	Introduction	6
1.1	The Zarafshan River Basin	6
1.2	Water System Modelling with TaqSim	10
2	System Model	13
2.1	Network Topology	13
2.2	Node Categories	15
2.3	The Two Reservoirs	16
2.4	Agricultural Demand Nodes	17
2.5	The Navoi Thermal Power Plant	18
2.6	Terminal Sinks and Interbasin Transfers	18
3	Historical Data	21
3.1	Data Sources and Seasonal Patterns	21
3.2	Canal Properties and Loss Configuration	23
4	Baseline Simulations	25
4.1	Uniform Baseline	25
4.1.1	Resulting Water Balance	26
4.2	Static-Tuned Baseline	31
4.3	Monthly-Varying Baseline	35
4.4	Comparison of Baseline Strategies	38
4.4.1	Progression Summary	39
4.4.2	The Role of Canal Losses	42
5	Conclusions and Outlook	46
5.1	What Allocation Can Achieve	46
5.2	What Allocation Cannot Achieve	46
5.3	From Simulation to Optimisation	46
5.4	Model Assumptions and Limitations	47
6	Exercises	49
6.1	Discussion Questions	49
6.2	Review Questions	50
7	Appendix A: Canal Loss Calibration Methodology	51
7.1	The Loss Model	51
7.2	Calibration Targets	52
7.3	Calibration Procedure	52

7.4	Calibrated Coefficients	52
7.5	Limitations	53
8	Appendix B: The TaqSim Simulation Framework	54
8.1	The Simulation Loop	54
8.2	Node Types	55
8.3	Reaches, Edges, and Water Transport	56
8.4	Event System and Traces	57
8.4.1	Common Event Families	57
8.4.2	Querying Events	58
8.4.3	Traces	58
8.5	Strategies: Operational Policies vs Physical Models	59
8.5.1	The Strategy Base Class	60
8.5.2	Discovery and Validation	60
8.5.3	Constraints	61
8.6	Parameter Exposure for Optimisation	61
8.6.1	ParamSpec	61
8.6.2	The Vectorisation API	61
8.6.3	Time-Varying Parameter Expansion	62
8.6.4	Constraint Repair	62
8.7	Objectives: Evaluating System Performance	63
8.7.1	Built-In Objectives	63
8.7.2	The <code>lift</code> Decorator	63
8.7.3	ZRB Application	63
8.8	Multi-Objective Optimisation	64
8.8.1	Dominance and the Pareto Front	64
8.8.2	NSGA-II via <code>ctrl-freak</code>	64
8.8.3	The <code>ctrl_freak.nsga2()</code> API	65
9	Appendix C: <code>zarafshan_taqsim</code> Module Reference	67
10	Appendix D: Glossary	69
	References	71

i Learning Objectives

After completing this coursebook, you should be able to:

1. **Describe** the topology and components of a node-edge water system model, including source, storage, demand, splitter, pass-through, and sink nodes.
2. **Interpret** a water balance table and explain what each component (inflow, consumption, losses, storage change, closure error) represents physically.
3. **Compare** allocation strategies using demand-aware efficiency metrics (effective efficiency, waste fraction) and explain why naive metrics like system efficiency can be misleading.

4. **Analyse** the impact of canal transmission losses on system performance and identify priority infrastructure investments from edge-level loss rankings.
5. **Use** the TaqSim simulation framework and the `zarafshan_taqsim` package to build, run, and analyse water system scenarios programmatically.
6. **Explain** TaqSim’s event-sourced simulation architecture—how the simulation loop, strategy system, and parameter exposure work together to enable reproducible analysis and automated optimisation (see Section 8).
7. **Identify** the foundations for multi-objective water system optimisation, including how objectives, Pareto fronts, and the NSGA-II algorithm connect simulation outputs to trade-off analysis.

Prerequisites

This coursebook assumes the following background:

- **Mathematical:** Basic algebra, ratios and proportions, unit conversions between flow rates and timestep volumes (m^3/s to m^3/day , and monthly totals where needed)
- **Domain:** Introductory hydrology concepts (river discharge, evapotranspiration, water balance). If unfamiliar, see Groll et al. (2015) for Central Asian hydrology context.
- **Computational:** Working knowledge of Python (functions, dictionaries, list comprehensions), familiarity with pandas DataFrames and matplotlib/plotly plotting
- **Conceptual:** Directed graphs (nodes and edges). No prior experience with water system modelling or optimisation is required.

Units and Conventions

Symbol / Unit	Meaning
m^3/s	Flow rate (cubic metres per second)
m^3/day	Volume per daily model timestep = flow rate $\times$ 86,400 s
m^3/month	Monthly source-data total or monthly aggregate of daily outputs
Mm^3	Megacubic metres (10^6 m^3) — used for reservoir capacities and annual component volumes
Bm^3	Billion cubic metres (10^9 m^3) — used for total annual inflow and system-level volumes
S_d	Seconds per day = 86,400 s

S_m	Average seconds per month $\approx 2,630,016$ s (= 30.44 days $\times$ 86,400 s/day)
% Inflow	A water balance component expressed as a percentage of total system inflow

The executable model runs at a **daily timestep** (Timesteps = 2191, covering 2017–2022). River inflow is injected as a daily observed series. Several source datasets remain monthly and are replicated or disaggregated to daily model inputs where needed. Tables typically report annualized volumes in Mm³ or Bm³ for readability.

1 Introduction

1.1 The Zarafshan River Basin

The Zarafshan River (also spelled Zeravshan) is one of the more significant transboundary rivers in Central Asia, flowing through Tajikistan and Uzbekistan (Groll et al. 2015). Its name means “Gold-Strewing” in Persian, referring to the historical gold deposits along its banks. Stretching approximately 877 km from the Zarafshan Glacier in Tajikistan (2,775 meters above sea level) to its disappearance in the Kyzylkum Desert, the river supports a basin area of roughly 17,700 km² and has sustained irrigated agriculture for over 2,500 years (Karimov et al. 1995). The ancient cities of Samarkand and Bukhara—both UNESCO World Heritage Sites—were built along its banks and depend on its waters to this day.

The Zarafshan exhibits a *nivo-glacial* hydrological regime, meaning its flow is fed predominantly by snowmelt and glacial ice rather than rainfall (Shultz 1965). In the upper basin, the Matcha mountain catchment (4,550 km²) harbours 118 glaciers covering 270.7 km² of ice, which contribute approximately 22% of annual runoff basin-wide and up to 37% during the peak summer months of July–September. Rainfall is negligible as a direct runoff source, accounting for barely 1% of annual discharge at the Dupuli gauging station in Tajikistan—the principal hydrometric station at the entrance to the irrigated lowlands some tens of kilometers upstream of the point of river inflow into Uzbekistan. This glacial and nival dominance produces a strongly seasonal hydrograph: July alone delivers over 25% of annual flow, while March—before the snowmelt season begins—contributes less than 2%.

At Dupuli, mean discharge is approximately 155–160 m³/s. Remarkably, the Zarafshan’s interannual variability is among the lowest of any Central Asian river, with a coefficient of variation of only $C_v = 0.08$ (Shultz 1965). This stability—a consequence of the glacier-buffered runoff regime and an average catchment altitude of roughly 3,000 meters above sea level—means that even severe droughts produce comparatively modest flow reductions: during the 1917 Central Asian drought, Zarafshan discharge fell by only 10%, whereas the neighbouring Chirchik River declined by 40%. The basin benefits from the Hissar Range, whose southern slopes receive over 2,000 mm of precipitation annually, though Samarkand in the lowlands receives only 323 mm/yr—with 39% falling in December–January and just 1.5% in June–August. Below the irrigation offtake points in nowadays Uzbekistan, the natural hydrograph is progressively transformed: diversions strip the summer peak while irrigation return flows create an artificial winter peak, producing a distinctive dual-peaked annual cycle at downstream stations (Shultz 1965).

The modern Zarafshan faces a convergence of challenges that make it an ideal case study for water system modelling. The river is fully allocated; across the lowland plains—an area of approximately 11,800 km²—runoff losses to irrigation, seepage, and evaporation exceed

the rate of runoff formation by a factor of 1.5, and in dry years downstream users face severe shortages (Shultz 1965; Kulmatov et al. 2013). Tajikistan controls the headwaters while Uzbekistan has most agricultural demand, creating transboundary tensions (Groll et al. 2015). Soviet-era canals and reservoirs require modernisation, and glacier retreat under climate change threatens long-term water availability—particularly given the 22–37% glacial contribution to summer flow. Agriculture, urban supply, industry, and environmental flows all compete for the same finite resource.



Figure 1.1: GIS map of the Zarafshan River Basin. The river and its branches is shown in blue color. Flow direction is from right to left. The main manmade control structures for water diversions and storage are highlighted with orange circles along the river stretches and canals. The two reservoirs are also shown and located in the midstream. Source: Giovanoli (2025b)

Figure 1.1 shows the downstream part of the Zarafshan River Basin that serves as our study area. The map highlights the study area boundary, rivers, canals, the two reservoirs (Kattakurgan and Akdarya), irrigated areas, and the Navoi thermal power plant. The key hydroworks—diversion structures that split river flow among competing branches—are highlighted in orange. The irrigated lowlands are subdivided into administrative irrigation districts spanning multiple oblasts (Samarkand, Navoi, Kashkadarya, Jizzakh); a detailed colour-coded district map is provided in Figure 2.1.

Figure 1.2 shows the engineering schematic of the magistral canal system, provided by the Uzbek Ministry of Water Resources [MoWR; Giovanoli (2025b)]. Magistral (main) canals appear in bright blue, with secondary and tertiary distribution canals visible as thinner branches. The numbered nodes 1–7 correspond to the key infrastructure components used in the simulation model: seven HydroWorks diversion structures and two reservoirs. The two nodes labelled “8” mark the interbasin transfer points to the Jizzakh and Kashkadarya regions, shown as grey-shaded insets.

Following the flow from east to west (right to left in the figure), water enters at the

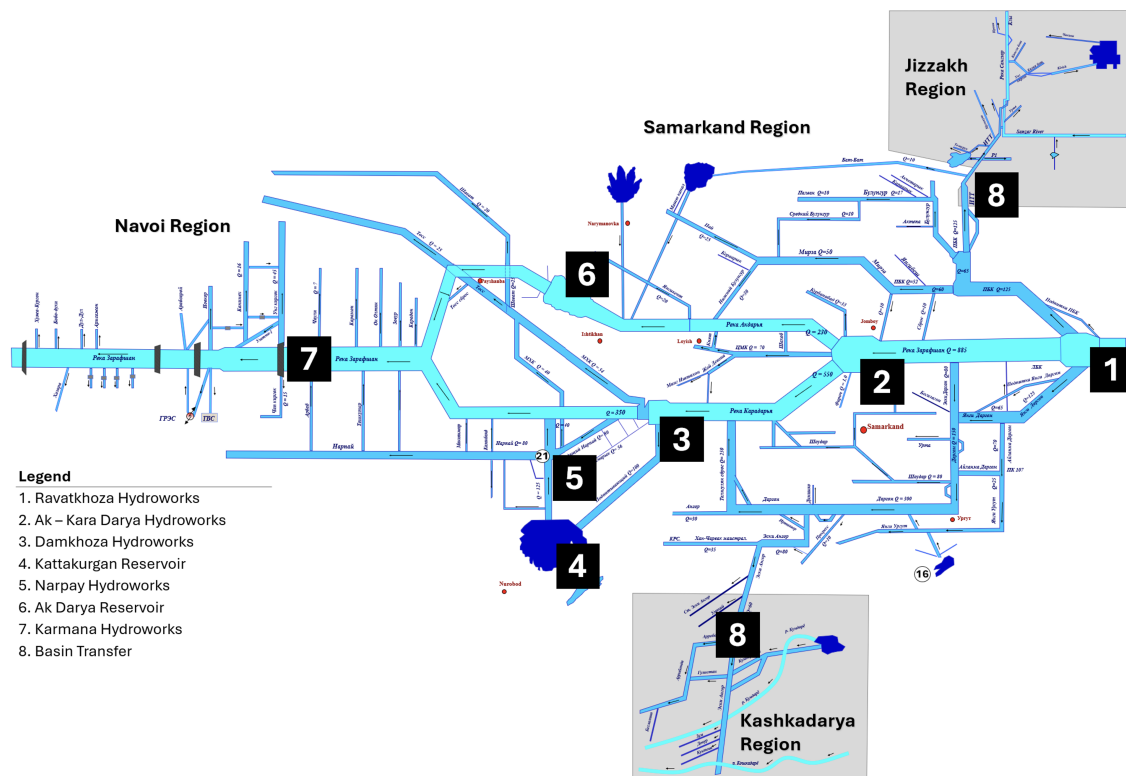


Figure 1.2: Magistral canal system of the Zarafshan River Basin, showing design capacities (Q , m^3/s) at key infrastructure nodes. Numbered nodes: (1) Ravatkhoza, (2) Ak-Kara Darya, (3) Damkhoza, (4) Kattakurgan Reservoir, (5) Narpay, (6) Akdarya Reservoir, (7) Karmana Hydroworks, (8) interbasin transfer points to Jizzakh and Kashkadarya (grey insets). Source: Uzbek Ministry of Water Resources via Giovanoli (2025b).

Ravatkhoza Hydroworks (node 1), which is the first major diversion point on the Zarafshan River. Here the flow splits into the main river channel continuing to Ak-Kara Darya ($Q = 885 \text{ m}^3/\text{s}$ design capacity), the Mirzapay and Dargom canals ($Q = 125 \text{ m}^3/\text{s}$ each), and the Jizzakh interbasin transfer ($Q = 45 \text{ m}^3/\text{s}$). At the **Ak-Kara Darya Hydroworks** (node 2), the main river divides further into the Damkhoza branch ($Q = 550 \text{ m}^3/\text{s}$), the Akkaradarya canal ($Q = 120 \text{ m}^3/\text{s}$), and a feeder to the **Akdarya Reservoir** (node 6, $Q = 230 \text{ m}^3/\text{s}$). The **Damkhoza Hydroworks** (node 3) distributes flow along the Karadarya river ($Q = 350 \text{ m}^3/\text{s}$), the Narpay canal ($Q = 80 \text{ m}^3/\text{s}$), the Miankaltoss canal ($Q = 100 \text{ m}^3/\text{s}$), and into **Kattakurgan Reservoir** (node 4, $Q = 100 \text{ m}^3/\text{s}$). Downstream, the **Narpay Hydroworks** (node 5) and the two reservoirs provide mid-system regulation before flow reconverges at the **Karmana Hydroworks** (node 7), which distributes water to the Navoi region ($Q = 400 \text{ m}^3/\text{s}$) and the relevant Navoi thermal power plant (Figure 1.3).



Figure 1.3: Water intake structure at the Karmana Hydroworks diverting flow to the Navoi thermal power plant. Cooling towers of the power plant are shown in the distance. Source: Tobias Siegfried, hydrosolutions GmbH.

This coursebook investigates a central question: **how much does the allocation rule at these diversion points matter?** Agricultural irrigation demand alone accounts for roughly 24% of total system inflow—but the basin must also serve substantial interbasin transfer obligations to Bukhara (via Navoi), Jizzakh, and Kashkadarya, which together raise total

obligations to approximately 53% of inflow. After accounting for canal transmission losses through a predominantly earthen canal network, the system is genuinely water-constrained. The challenge is twofold: *distribution*—getting the right amount of water to the right place at the right time—and *efficiency*—minimising losses in a canal network that loses a significant fraction of flow to seepage and evaporation. We build the system model (Section 2), examine historical supply and demand patterns (Section 3), then compare three increasingly sophisticated allocation strategies (Section 4), including the role of canal transmission losses (Section 4.4.2).

1.2 Water System Modelling with TaqSim

TaqSim is an event-sourced water system simulation framework (Lazaro and Siegfried 2024) (see Section 8 for a detailed architectural reference). Unlike traditional models that track state variables directly, TaqSim records **events**—immutable records of water movement (water received, released, stored, lost)—that can be replayed and analysed after simulation. This architecture enables reconstruction of any system state at any timestep, pattern analysis across scenarios, and the construction of custom metrics from event traces without re-running simulations.

The core abstractions in TaqSim are straightforward. A **WaterSystem** is a directed graph of **nodes** connected by **edges**. Nodes represent physical components—reservoirs, irrigation demands, diversion points—while edges represent water conveyance infrastructure (canals, pipes) with capacity limits and loss properties. **Strategies** are rules that govern node behaviour—how much water a reservoir releases, and how a diversion node splits flow among its downstream branches (the distinction between operational strategies and physical models is explained in Section 8.5). The choice of splitting strategy is the central variable in this coursebook: we compare uniform splitting (equal shares to all branches), static demand-proportional splitting (one fixed ratio based on long-term demand), and a time-varying demand-aware split rule whose seasonal structure is applied on the daily simulation clock.

After simulation, TaqSim’s event log enables a comprehensive **water balance**—a mass-conservation accounting that tracks every cubic metre from inflow through to consumption, transmission losses, storage changes, and terminal outflow. The water balance is the primary analytical tool throughout this coursebook, used to compare strategies and diagnose where water is gained and lost. TaqSim also provides **objectives**—functions that evaluate system performance for **multi-objective optimisation** (simultaneously minimising multiple competing goals such as agricultural deficit and reservoir spillage). The companion optimisation coursebook uses NSGA-II, an evolutionary algorithm, to find **Pareto-optimal** solutions—configurations where no objective can be improved without worsening another (see Section 8.7 for the objective system and Section 10 for definitions).

How Event-Sourcing Works

To make the event-sourced concept concrete, consider a single canal edge carrying 50 m³/s for one daily timestep at $t = 150$ in 2017. TaqSim records three events on that edge:

Event	Amount	Meaning
<code>WaterReceived(t=150)</code>	4.32 Mm ³	Flow entering the canal that day
<code>WaterLost(t=150, reason='seepage')</code>	0.32 Mm ³	Seepage loss through canal bed
<code>WaterDelivered(t=150)</code>	4.00 Mm ³	Flow exiting the canal to the next node

No running state variable is updated—each event is an immutable record. After simulation, `compute_water_balance()` sums all `WaterLost` events (by reason) and all `WaterReceived` events (by node type) to reconstruct the full system balance. This is why you can compare strategies by simply re-querying the event log with different aggregation functions, without re-running the simulation.

TaqSim combines several functional node types with transport segments to model a water system (detailed teaching descriptions appear in Section 8.2):

Node Type	Behaviour
Source	Generates water (river inflows, runoff)
Storage	Stores water with a release rule (reservoirs, lakes)
Demand	Consumes water, records deficit (irrigation, cities)
Splitter	Divides flow among multiple downstream paths
PassThrough	Conveys water with capacity limit (canals, turbines)
Sink	Terminal node where water exits the system

The `zarafshan_taqsim` Python package implements this system model. Its six modules—for data loading, operating strategies, network assembly, water balance analysis, visualisation, and optimisation objectives—are documented in Section 9. The underlying TaqSim framework architecture is documented in Section 8.

Exercise A: Conceptual – Sketch a Minimal Water System

Task: On paper (or in a drawing tool), sketch a minimal water system with exactly 4 nodes: one Source, one Storage (reservoir), one Demand (irrigation district), and one Sink. Draw directed edges showing how water flows from source to sink.

1. How many edges does your system have? Is there more than one valid topology?
2. If the Source generates 100 m³/s and the Demand requires 60 m³/s with 80%

efficiency, how much water does the Demand node withdraw? How much reaches the Sink?

3. If the reservoir is full, what happens to incoming water that cannot be stored?

Hint: The Demand node withdraws $\text{requirement} / \text{efficiency} = 60 / 0.8 = 75 \text{ m}^3/\text{s}$. The remaining $25 \text{ m}^3/\text{s}$ passes through to the Sink. Reservoir overflow is recorded as a loss event.

Expected result: Your system should have 3 edges (Source→Storage→Demand→Sink is one valid topology; Source→Demand→Storage→Sink is another). The Demand withdraws $75 \text{ m}^3/\text{s}$ and $25 \text{ m}^3/\text{s}$ passes through. Multiple valid topologies exist depending on where you place the reservoir.

! Key Takeaways – Introduction

- The Zarafshan River is fully allocated: total obligations (irrigation + interbasin transfers) reach ~53% of inflow, leaving limited margin after canal losses.
- TaqSim is an event-sourced simulation framework: it records immutable events (water received, lost, stored) that enable flexible post-simulation analysis.
- A water system is modelled as a directed graph of functional nodes (Source, Storage, Demand, Splitter, PassThrough, Sink) connected by transport segments that move water and can incur losses.
- The central question: **how much does the splitting rule at diversion points matter?**

2 System Model

2.1 Network Topology

The network topology of the simulation model is oriented around two primary data sources: the available canal network infrastructure and the administrative division of the basin into irrigation districts. The magistral canal schematic provided by the Uzbek Ministry of Water Resources [MoWR; Giovanoli (2025b)] defines the physical backbone of the system—the main canals, their branching structure, design capacities, and the locations of the hydroworks diversion structures that control flow distribution (see Figure 1.2). The second orienting reference is the partition of the irrigated lowlands into administrative irrigation districts, as shown in Figure 2.1.

The colour-coded district boundaries in Figure 2.1 reveal how the Zarafshan’s water is partitioned among competing administrative regions. Each major irrigation district corresponds to a service area fed by one or more magistral canals branching off at a hydroworks diversion point. In the simulation model, these districts are aggregated into six Demand nodes, each representing the combined water requirement—conveyance losses, field application losses, and crop evapotranspiration—of an entire irrigation district. The district-level aggregation is a deliberate modelling choice: it matches the spatial scale at which allocation decisions are made at the hydroworks and at which demand data are available from remote sensing (Ragettli et al. 2025). The map also makes visible the interbasin transfer obligations to Jiz-zakh (northeast) and Kashkadarya (south), which are served by dedicated canals branching off the main network and modelled as Sink nodes in the simulation.

Combining these two data sources, the ZRB model represents the water system as a directed graph of functional water-management nodes connected by transport segments. Water enters at seven Source nodes—one representing the Zarafshan River at the Ravatkhoza gauging station and six representing precipitation-driven local runoff at each irrigation district’s catchment. It then moves through a branching network of HydroWorks diversion points (Splitter nodes), two reservoirs (Storage nodes), six agricultural irrigation districts (Demand nodes), one thermal power plant (PassThrough node), and three terminal Sink nodes. Between these functional nodes, the model includes transport segments that represent canals and river reaches and account for conveyance losses.

Figure 2.2 displays the complete network, with node colours indicating type. The topology captures the essential structure of the ZRB’s water distribution infrastructure (cf. the real-world canal schematic in Figure 1.2 and the irrigation district map in Figure 2.1): a single source that branches progressively into sub-networks serving different regions, with two reservoirs providing seasonal buffering. Each edge carries estimated physical properties—lining type (earthen, partial, or concrete), canal condition, length, and width—that determine



Figure 2.1: Irrigation map of the Zarafshan Valley (), produced by the USAID Central Asia Natural Resources Management Project (NRMP). Irrigation districts are colour-coded by oblast: Navoi (green), Samarkand (pink), Kashkadarya (yellow), Jizzakh (peach), and Leninabad—now Sughd Province, Tajikistan—(purple). Blue lines trace the magistral canal network carrying water from the Zarafshan River through successive branching diversions; cyan lines indicate collector drains that gather return flows and subsurface drainage. Orange symbols mark hydroworks structures (circles) and pumping stations. Irrigated areas appear as shaded patches within each district. The map illustrates how the basin’s water distribution infrastructure spans multiple administrative boundaries and how individual irrigation districts—such as Dargom, Mirzapay, Narpay, Miankalt-Toss, Akkaradarya, and Iski-Angar—are served by dedicated offtake canals branching from the main stem. Source: USAID (n.d.), reproduced via Giovanoli (2025b).

2.3 The Two Reservoirs

The ZRB has two major reservoirs that provide seasonal storage (Ragettli et al. 2025; CAWater-Info 2024a). **Kattakurgan Reservoir** (capacity 667 Mm³, built 1965) serves as multi-year storage for drought buffering. **Akdarya Reservoir** (capacity 106 Mm³, built 1967) provides seasonal regulation that, among other benefits, helps ensure reliable water supply to the downstream Navoi thermal power plant (Rakhmatullaev et al. 2011). Both reservoirs have dead storage set at 10% of capacity, representing water below intake structures that cannot be released.

Reservoir operations are governed by a Storage Level Operating Policy (SLOP) with four zones: dead storage (below 10%, no release possible), a buffer zone (reduced release to conserve water), a conservation zone (release at the target rate), and a flood control zone (release excess to prevent uncontrolled spillage). The default target release rate v_r is defined from a monthly capacity-turnover heuristic: $v_r = (C \times 0.20)/S_m$, where C is reservoir capacity and S_m is average seconds per month, additionally capped at the total capacity of downstream outlet infrastructure. This corresponds to approximately 20% of capacity per average month—a sustainable operating heuristic that prevents small reservoirs from draining too quickly while providing sufficient turnover for large reservoirs—without exceeding what the outlet works can physically convey. These release parameters are monthly-cyclical, but the model itself executes on a daily timestep.

i SLOP Zones Illustrated

The four SLOP zones map directly to reservoir storage level:

Storage (%)		
100	Capacity	
	FLOOD CONTROL	Release > v_r (shed excess)
90		
	CONSERVATION	Release = v_r (target rate)
30		
	BUFFER	Release < v_r (conserve)
10		
	DEAD STORAGE	Release = 0 (physically impossible)
0		

Worked example (Kattakurgan): Capacity $C = 667$ Mm³. Dead storage = 67 Mm³ (10%). Suppose the current storage is 400 Mm³ (60% of capacity)—this falls in the **conservation zone** (between 30% and 90%), so the reservoir releases at the target rate v_r . The default $v_r = (667 \times 10^6 \times 0.20)/2,630,016 \approx 50.7$ m³/s, corresponding to about 133 Mm³ over an average month. In the daily simulation, that release rate is converted to daily volumes at each timestep. If storage drops to 130 Mm³ (20%), the reservoir enters the **buffer zone** and reduces release to conserve water. If storage drops below 67 Mm³, the reservoir is in **dead storage** and releases nothing.

Table 2.2: Reservoir configurations showing total capacity, dead storage, active storage, and initial storage levels.

Table 2.2

Reservoir	Total Capacity (Mm ³)	Dead Storage (Mm ³)	Active Storage (Mm ³)	Initial Storage (Mm ³)
Kattakurgan	667	67 (10%)	600 (90%)	334 (50%)
Akdarya	106	11 (10%)	95 (90%)	53 (50%)

2.4 Agricultural Demand Nodes

The ZRB serves six major irrigation districts as shown in Figure 2.1, each modelled as a TaqSim Demand node. Water demand at these nodes has been derived from regional assessment of irrigation water demand using remote sensing (Ragettli et al. 2025). A demand node abstracts an entire irrigation district as defined in Figure 2.1 with primary canal offtakes, internal secondary and tertiary canal networks, field-level infrastructure, and crop water consumption (evapotranspiration). Each district’s demand node computes withdrawal needed as **requirement** / **efficiency**, where the combined efficiency is the product of conveyance efficiency (fraction surviving the district’s internal canals) and field efficiency (fraction reaching crop roots). The node only withdraws what it needs; excess water passes through to downstream nodes.

Importantly, demand nodes in TaqSim are **pass-through**: water delivered in excess of a district’s requirement continues downstream to subsequent nodes. This creates cross-connections in the network—for example, excess water at Dargom flows onward to HW_Damkhoza, and excess at Mirzapay reaches RES_Akdarya—meaning that over-delivery to one district is not entirely wasted but may benefit downstream users. Section 1.2 explains in greater detail the concept of a pass-through node.

This behaviour has an important analytical consequence: water delivered *beyond* a district’s requirement in any given month provides no additional benefit at that district—it simply passes through. We call this **over-delivery**, and the complementary concept of **effective delivery** counts only water arriving up to each district’s demand per timestep. These metrics become central to the strategy comparison in Section 4.4, where they reveal that naive allocation can flood some districts while starving others of water at the same time.

Table 2.3 lists the six irrigation districts and their efficiency parameters. These values are drawn from the FAO literature on irrigation efficiencies (FAO 1989) and adapted for Uzbekistan conditions. Needless to say that these somewhat generic values can be better tailored to local conditions and individual district characteristics if better information becomes available.

Table 2.3: Irrigation district configurations showing conveyance efficiency, field efficiency, and their combined product.

Table 2.3

District	Conveyance Eff.	Field Eff.	Combined
Dargom	65%	75%	48.8%
Mirzapay	65%	75%	48.8%
Akkaradarya	65%	75%	48.8%
Miankaltoss	65%	75%	48.8%
Narpay	65%	75%	48.8%
Karmanakonimex	65%	75%	48.8%

2.5 The Navoi Thermal Power Plant

The Navoi Thermal Power Plant (see Figure 1.3) is a critical industrial facility in the downstream section of the ZRB (Global Energy Monitor 2024). Unlike agricultural demands that consume water through evapotranspiration, the power plant requires non-consumptive cooling water—water passes through the cooling system and returns to the river. It requires a minimum flow of $15 \text{ m}^3/\text{s}$ (approximately $1.30 \text{ Mm}^3/\text{day}$ on the daily model clock) to operate safely (Giovanoli 2025a). Without adequate cooling water, turbines may overheat, power generation must be curtailed, and equipment damage may occur. This makes the power plant a hard constraint for water delivery: unlike agricultural deficits which reduce crop yields gradually, cooling water shortfalls cause immediate operational problems.

In TaqSim, the power plant is modelled as a PassThrough node with a capacity of $30 \text{ m}^3/\text{s}$ and a minimum flow of $15 \text{ m}^3/\text{s}$. The system dashboards in Section 4 track powerplant flow against this threshold at each simulation timestep, revealing how different allocation strategies affect cooling water reliability. In the companion optimisation coursebook, a dedicated objective function (see Section 8.7) enables formal trade-off analysis between powerplant reliability and other competing goals.

2.6 Terminal Sinks and Interbasin Transfers

The three terminal Sink nodes in the ZRB model—Navoi, Jizzakh, and Kashkadarya—represent water that leaves the modelled domain (Ragettli et al. 2025). The two interbasin transfers—to Jizzakh and Kashkadarya—are visible as grey-shaded regions in Figure 1.2. Unlike losses (which remove water unproductively), sink outflows are **deliberate commitments**: they supply populations, agriculture, and ecosystems beyond the study area boundary.

Sink	Destination	Purpose	Key constraint
Sink_Navoi	Bukhara oasis (downstream)	Irrigated agriculture, urban supply (Karimov et al. 1995)	Also serves the Navoi powerplant (Section 2.5); doubly sensitive to allocation
Sink_Jizzakh	Jizzakh province (Syrdarya basin)	Interbasin transfer for irrigation (CAWater-Info 2024b)	Governed by inter-provincial agreements; not negotiable short-term
Sink_Kashkadarya	Kashkadarya province (south)	Interbasin transfer for irrigation (CAWater-Info 2024b)	Local river insufficient; transfer is a structural necessity

These three sinks collectively account for a significant share of total system throughput—often comparable in magnitude to district consumption (see water balance tables in Section 4). Any allocation strategy that reduces sink outflow is implicitly redistributing water away from downstream and neighbouring provinces. From a modelling perspective, the sinks serve as **boundary conditions** with minimum flow requirements encoding the lower bound on acceptable outflow. Differences in sink outflow across strategies reveal how each balances upstream demand against interbasin obligations—a trade-off that ultimately requires political negotiation, not just hydraulic optimisation.

💡 Exercise 1: Explore the Network

Using the `load_edges()` function, find **all distinct paths** from **Source** to **Sink_Navoi**. **Task:** Write code to load the edge configuration and enumerate all routes from **Source** to **Sink_Navoi**. How many distinct routes exist? Which path passes through the most nodes?

Hint: You will need to import the function: `from zarafshan_taqsim import load_edges`. The edge data is a dict mapping each node to its list of downstream targets. A recursive depth-first search works well here. Watch out for nodes that appear as targets of multiple upstream nodes.

Expected result: You should find multiple distinct paths (on the order of 5–10). The longest path passes through 7–8 nodes. Paths diverge at the first HydroWorks splitter and reconverge via demand node pass-through connections.

! Key Takeaways – System Model

- The ZRB model combines sources, splitters, reservoirs, demand nodes, a downstream powerplant pass-through, terminal sinks, and intervening transport segments that convey water and incur losses.

- Reservoir operations use a four-zone SLOP: dead storage (no release), buffer (reduced), conservation (target rate), flood control (excess release).
- Demand nodes are **pass-through**: excess water continues downstream, creating network interdependencies.
- The three sinks represent interbasin transfer obligations to Bukhara, Jizzakh, and Kashkadarya—not waste, but commitments.

With the system topology defined, we now turn to the historical data that drives it: how much water enters the system, when demand peaks, and what the canal infrastructure looks like.

3 Historical Data

3.1 Data Sources and Seasonal Patterns

The daily simulation covers the 2017–2022 period. River inflow is injected from a daily observed series at the Ravatkhoza gauging station, while irrigation demand, precipitation, evaporation, and minimum environmental flow requirements are derived from the historical datasets assembled in Ragettli et al. (2025) and translated to daily model inputs where needed. The figure below is a **monthly source-data overview** used to summarize the seasonal structure of supply and obligations before turning to the daily simulation outputs.

Central Asian hydrology exhibits strong seasonal patterns (Groll et al. 2015). Spring (March–May) brings snowmelt and rising inflows. Summer (June–August) delivers peak inflows from snow and, later in the season, glacier melt coinciding with peak irrigation demand. Autumn (September–November) sees declining flows and the harvest season, while winter (December–February) has low flows and minimal demand.

Figure 3.1 summarizes the monthly source data that underpins the daily model. Total system inflow averages approximately $4.7 \text{ Bm}^3/\text{yr}$, of which $3.96 \text{ Bm}^3/\text{yr}$ enters via the Zarafshan at Ravatkhoza and $0.71 \text{ Bm}^3/\text{yr}$ arrives as distributed precipitation runoff at the six tributary catchments. Irrigation demand across all six districts totals roughly $1.14 \text{ Bm}^3/\text{yr}$ —only about 24% of inflow. However, the system must also fulfil substantial interbasin transfer obligations: $0.78 \text{ Bm}^3/\text{yr}$ to Navoi (Bukhara oasis), $0.34 \text{ Bm}^3/\text{yr}$ to Kashkadarya, and $0.19 \text{ Bm}^3/\text{yr}$ to Jizzakh (Syrdarya basin). Together, **total obligations reach approximately $2.5 \text{ Bm}^3/\text{yr}$, or 53% of inflow**. The remaining 47% must cover canal transmission losses (seepage, evaporation, operational spillage), reservoir evaporation, and storage changes—leaving little margin for error in how water is allocated once the system is simulated on the daily timestep.

The Figure reveals that water stress can occur in spring and late summer time at the system level. This requires careful management of the waters of the river, including the timing of the delivery as well as the operation of the two reservoirs.

Exercise B: Supply–Demand Arithmetic

Using the numbers from Figure 3.1, answer the following:

1. If canal transmission losses consume approximately 35% of total inflow, how much water (in Bm^3/yr) is lost to canals? Is this more or less than total irrigation demand?
2. The combined reservoir capacity (Kattakurgan + Akdarya) is approximately 773

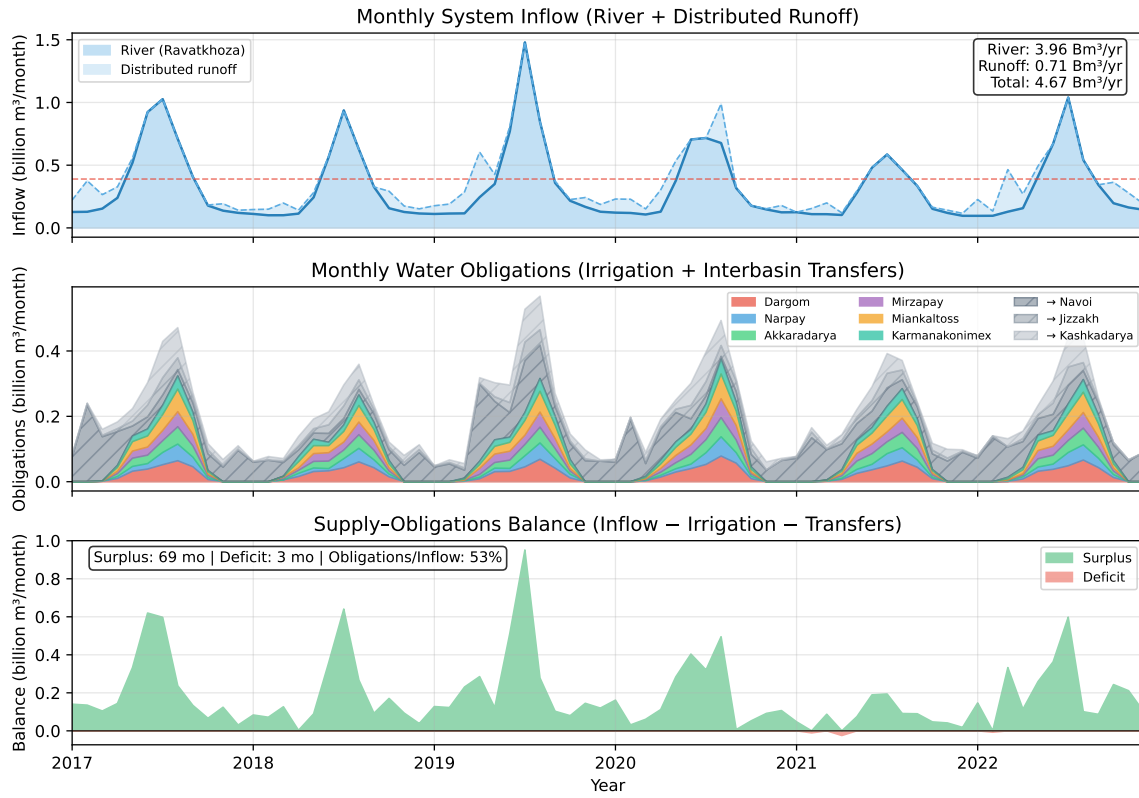


Figure 3.1: Monthly source-data overview of supply and obligations. Top: stacked total system inflow comprising river flow at Ravatkhoza and distributed precipitation runoff from six tributary catchments. Middle: stacked water obligations including irrigation demand (six districts, coloured) and interbasin transfer requirements (three sinks, hatched grey). Bottom: supply–obligations balance where green indicates surplus months and red indicates deficit months.

Mm³. Express this as a fraction of total annual inflow. How many months of average inflow could the reservoirs store?

3. If you could eliminate all canal losses, would total inflow be sufficient to meet all obligations (irrigation + interbasin transfers) in every month of the source-data series? Why or why not?

Hint: Total inflow is ~4.7 Bm³/yr. For question 3, consider that gross withdrawal needs (requirement / efficiency) are roughly double the crop water requirement, and that multiple districts and sinks compete for the same flow simultaneously.

Expected result: (1) Canal losses ~1.6 Bm³/yr, which exceeds total irrigation demand (~1.14 Bm³/yr). (2) Reservoir capacity is ~16% of annual inflow, or roughly 2 months of average inflow. (3) No—even without losses, total obligations are ~53% of inflow, and gross withdrawal needs at district gates are roughly double the crop requirement due to conveyance inefficiency; in peak months, competing withdrawals across the network can exceed available supply.

3.2 Canal Properties and Loss Configuration

The ZRB canal network is represented by many transport segments connecting the functional nodes. Each segment has physical properties that determine transmission losses: length (km), average water surface width (m), lining type (earthen, partial, or concrete), and infrastructure condition (good, average, or poor). The network topology and canal capacities are derived from the magistral canal schematic shown in Figure 1.2, provided by the Uzbek Ministry of Water Resources [MoWR; Giovanoli (2025b)].

The design capacities of the canals in the network are taken from Figure 1.2. They are the maximum flow each magistral canal segment is engineered to carry. These are infrastructure limits, not observed flows. The simulation model uses these design capacities as edge capacity constraints, ensuring that simulated flow cannot exceed what the physical infrastructure can convey.

However, **per-canal physical properties used for loss modelling are estimated values:** lengths were measured from GIS, widths were scaled from design flow capacity, lining types were assigned based on local knowledge and infrastructure surveys, resulting in approximately 63% earthen canals—somewhat below the national average of 75% (Khodjiyev and Atadjanova 2022), reflecting that the main trunk canals in the Zarafshan system are partially lined, and conditions were defaulted to “average” in the absence of field survey data. Overall network efficiency is approximately 63% (Khodjiyev and Atadjanova 2022). Canal losses are analysed in Section 4.4.2.

As Figure 3.2 shows, the majority of canals in the network are earthen (unlined), which is the primary driver of seepage losses. The scatter plot reveals that earthen canals tend to be both longer and wider than lined canals, compounding the loss problem.

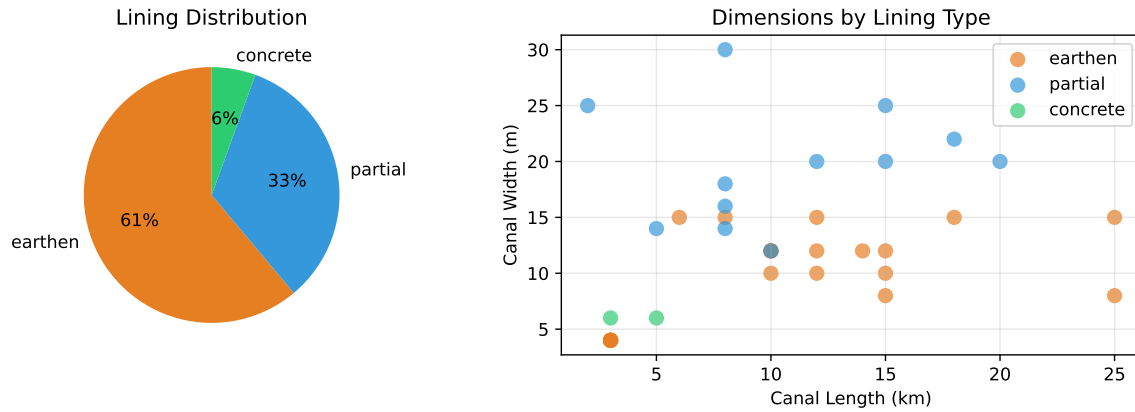


Figure 3.2: Canal infrastructure properties: lining type distribution (left) and canal dimensions by lining type (right). The dominance of earthen canals explains the high transmission losses observed in the system.

! Key Takeaways – Historical Data

- Total inflow averages $\sim 4.7 \text{ Bm}^3/\text{yr}$; total obligations (irrigation + interbasin transfers) are $\sim 2.5 \text{ Bm}^3/\text{yr}$ (53% of inflow).
- Strong seasonality: glacier and snowmelt drive summer supply peaks that broadly coincide with peak irrigation demand—but the system is over-committed, so reservoirs must buffer months when competing withdrawals and losses exceed available flow.
- The canal network is predominantly earthen ($\sim 63\%$), with canal properties estimated from GIS and literature rather than field surveys.
- The system is genuinely water-constrained: after losses, there is limited margin for allocation error.

The supply–demand analysis confirms that the ZRB is water-constrained: obligations exceed 50% of inflow, and canal losses consume another third. The question is no longer *whether* allocation matters, but *how much*. We now test three increasingly sophisticated splitting strategies on this system.

4 Baseline Simulations

With the system topology, data inputs, and canal infrastructure established, we now examine how water flows through the network under different operating rules. We proceed through three increasingly sophisticated splitting strategies for the distribution of available flows throughout the system—uniform, static-tuned, and monthly-varying—before comparing them side by side. All simulations include realistic canal losses throughout the transport network. This helps us to get a good understanding of the water system and how different allocation rules determine outcomes in terms of demand satisfaction.

In a companion optimisation coursebook, we then take this water system and study different water allocation patterns that are optimized against a set of objectives to be defined.

4.1 Uniform Baseline

The uniform baseline uses equal splitting at every HydroWorks distribution node. For example, HW_Ravatkhoza splits inflow equally among its four downstream branches, regardless of the fact that some branches serve large irrigation districts while others feed low-demand sinks (see the network in Figure 1.2). This is the simplest possible operating rule and serves as a reference point. It is also quite a dumb operating rule as we shall see in due time. In terms of degrees of freedom, the uniform rule has **zero free parameters**: every ratio is locked at $1/n$ (where n is the number of outgoing branches), so there is nothing to tune. This makes it a useful null hypothesis, but it also means the rule is completely insensitive to the structure of downstream demand.

The following code block walks through the complete simulation-analysis workflow. Subsequent simulations follow the same pattern and are shown with code folded.

```
# Step 1: Create the water system with canal losses enabled
system = create_zrb_system(use_canal_losses=True)

# Step 2: Run the simulation over the full 2017-2022 daily period
system.simulate(TIMESTEPS)

# Step 3: Define the nodes we want to analyse
demand_ids = list(DEFAULT_DISTRICTS)          # 6 irrigation districts
reservoir_ids = ["RES_Kattakurgan", "RES_Akdarya"]
POWERPLANT_MIN_FLOW = 15.0                    # m³/s minimum threshold
```

```
# Step 4: Reconstruct the water balance from TaqSim's event log
wb = compute_water_balance(system)
```

4.1.1 Resulting Water Balance

The water balance table is a mass balance—it tracks every drop of water that enters the system and accounts for where it ends up. Think of it as a financial ledger: inflows are income, beneficial use and losses are expenditures, and the closure error tells you whether the books balance.

Table 4.1 shows the water balance for the uniform baseline:

The individual terms are as follows:

River + Runoff is all water entering the system: the Zarafshan river inflow at the upstream boundary plus lateral runoff from precipitation at the six tributary source nodes. This total is the denominator for all “% Inflow” values in the table.

Under **Beneficial Use**, “Consumed by districts” is water actually removed from the hydrological system by the six agricultural demand nodes—the water that irrigates crops and evapotranspires. “Sink outflow” is water that reaches the three terminal Sink nodes, including the two interbasin water transfers and the outflow sink at Navoi at the downstream boundary that ensures water flows to the Bukhara region. Sink outflow is not a loss; it represents environmental flows, downstream commitments, or water leaving the modelled domain.

The **Losses** section accounts for water removed from the system unproductively. Canal seepage—water lost through canal beds and banks to groundwater—is computed with a $\sqrt{Q}$ model and is the dominant loss mechanism, as expected for a network of mostly earthen canals (Zaman et al. 2023). See Section 4.4.2. Reservoir evaporation is computed monthly from surface area (derived from height-volume curves) and open-water evaporation rates (Shibuo et al. 2007); note that reservoir evaporation rates are higher than canal evaporation rates because open-water bodies have greater fetch and exposure than narrow canal surfaces. Demand inefficiency represents field-level losses at demand nodes: each TaqSim Demand node withdraws **requirement** / **efficiency** but consumes only **requirement**; the difference is recorded as demand inefficiency loss. Canal evaporation and operational losses are minor components, and edge overflow captures water lost when flow exceeds a canal’s carrying capacity.

Reservoir Δ Storage is the net change in water stored over the simulation period ($S_{\text{final}} - S_{\text{initial}}$). Over a multi-year simulation, this should be small. The **Closure error** ($\text{Inflow} - \text{Consumed} - \text{Sink outflow} - \text{Losses} - \Delta\text{Storage}$) confirms mass conservation: a value near zero indicates the simulation correctly accounts for all water.

Table 4.2 shows the demand satisfaction under the uniform baseline operating scenario.

Table 4.1: Average annual water balance for the uniform baseline simulation, showing where every drop of water ends up over the full 2017–2022 daily simulation period.

Average Annual Water Balance (2017–2022)

	Total (Mm ³)	Annual (Mm ³)	% Inflow

Inflows			
River + Runoff	28,146.7	4,692.2	100.0%
Beneficial Use (mass balance)			
Consumed by districts (crop use)	4,852.8	809.0	17.2%
Sink outflow	9,275.9	1,546.3	33.0%
Losses			
Canal seepage	6,446.9	1,074.7	22.9%
Reservoir evaporation	671.3	111.9	2.4%
Demand inefficiency loss	5,101.6	850.5	18.1%
Canal evaporation	34.4	5.7	0.1%
Canal operational	282.9	47.2	1.0%
Edge overflow	1,740.0	290.1	6.2%
Total losses	14,277.1	2,380.1	50.7%
Storage			
Reservoir ΔStorage	-259.1	-43.2	-0.9%
Closure error	0.0	0.0	0.0%
Demand Context (not mass balance)			
Total agricultural demand	6,888.1	1,148.3	24.5%
Gross district delivery	22,534.9	3,756.7	80.1%
Effective delivery	5,484.0	914.2	19.5%
Over-delivery	17,050.9	2,842.5	60.6%

Demand satisfaction (effective delivery / total agricultural demand): 79.6%

Note: 'Consumed by districts (crop use)' is net consumptive use, not total demand.

Table 4.2: Per-district demand satisfaction under the uniform baseline, showing annual demand, delivery, deficit, and satisfaction percentage.

Per-District Annual Average: Demand, Supply, Deficit

District	Demand (Mm ³ /yr)	Delivered (Mm ³ /yr)	Deficit (Mm ³ /yr)	Satisfaction
Dargom	253.1	972.4	9.4	96.3%
Narpay	152.2	137.0	124.0	18.6%
Akkaradarya	188.1	622.2	19.8	89.5%
Mirzapay	179.8	1,096.1	0.9	99.5%
Miankaltoss	233.3	260.8	157.1	32.6%
Karmanakonimex	141.7	668.2	28.2	80.1%

Powerplant Flow Statistics

Min threshold:	15.0 m ³ /s
Mean flow:	8.7 m ³ /s
Min flow:	0.5 m ³ /s
Max flow:	29.9 m ³ /s
Steps below threshold:	1843 / 2191

Reading this table requires care, because at first glance some districts appear to receive far more water than they demand yet still show substantial deficits. Three factors explain this apparent paradox:

1. **Efficiency losses absorb most of the delivered water.** The “Demand” column is the *crop water requirement*—what plants actually need via evapotranspiration. But at a combined conveyance-and-field efficiency of only 48.8% (Table 2.3), the node must *withdraw* roughly twice the crop requirement from the canal to meet that need. For example, Dargom’s crops require 251.5 Mm³/yr, but the node must withdraw 251.5 / 0.488 = 515 Mm³/yr to deliver that amount to the root zone. The “Delivered” column shows total water flowing *into* the node—493.3 Mm³/yr—which falls short of the 515 Mm³/yr withdrawal need even on an annual basis.
2. **The allocation rule ignores seasonal demand patterns.** Although the Zarafshan’s nivo-glacial regime delivers peak supply in summer—broadly coinciding with peak irrigation demand—the uniform splitting rule sends a fixed fraction of river flow to each branch regardless of season. In winter, when demand is low, surplus water passes through to downstream nodes unused. In peak summer months, the fixed allocation fraction may be insufficient to cover the gross withdrawal need (requirement / efficiency) because canal losses are also at their highest ($\sqrt{Q}$ scaling). Because demand nodes have no storage, low-demand months cannot compensate for high-demand months.
3. **Spatial inequality under uniform splitting.** Equal splitting at every HydroWorks

ignores the vastly different demand magnitudes across branches. This creates extreme disparity: Mirzapay receives 994 Mm³/yr of inflow (needing only ~367 for full withdrawal) and achieves 99% satisfaction, while Narpay receives just 128 Mm³/yr (needing ~310) and manages only 15% satisfaction. The uniform rule floods some branches while starving others.

The powerplant flow statistics printed beneath Table 4.2 reveal an equally alarming situation for non-agricultural water use. The Navoi thermal power plant requires a minimum cooling water flow of 15 m³/s to operate safely (Section 2.5). Under uniform splitting, the mean flow reaching the plant is 12.6 m³/s—still below threshold—and the plant is under-supplied in **1465 out of 2191 daily timesteps**. The problem is structural: the Karmana Hydroworks (Figure 1.2, node 7), which feeds the powerplant branch, sits at the downstream end of the network. Under uniform splitting, each upstream diversion has already siphoned an equal share regardless of downstream needs, leaving the Karmana branch chronically under-supplied. This is a direct consequence of the “equal shares everywhere” rule applied to a network with highly unequal downstream obligations.

Unable to display output for mime type(s): text/html

- (a) Uniform baseline system dashboard (2017–2022 daily simulation). Panel 1: daily inflow, delivery, and losses. Panel 2: cumulative water balance. Panel 3: per-district annual demand vs. delivery. Panel 4: powerplant flow vs. the 15 m³/s minimum threshold.

Unable to display output for mime type(s): text/html

(b)

Figure 4.1

Figure 4.1 provides a four-panel diagnostic of the uniform baseline. An important clarification: “Delivery” in Panels 1 and 2 is the total water *received at the six agricultural demand nodes only*—it does **not** include outflows to the three terminal sinks (Navoi, Jizzakh, Kashkadarya). Sink outflows are a separate, substantial commitment (~29% of inflow) that competes with agricultural delivery for the same limited supply.

Panel 1 (Daily Inflow / Delivery / Losses) shows the daily time series over the full 2017–2022 simulation. The blue inflow line exhibits the characteristic nivo-glacial seasonal cycle: sharp summer peaks driven by snowmelt and glacier runoff and low winter baseflow. The green delivery line tracks how much water actually reaches the six irrigation districts each day. Under uniform splitting, delivery follows the same seasonal shape as inflow but at a much lower level. The gap between inflow and delivery is accounted for by three sinks: canal transmission losses (red line), sink outflows (not shown separately), and reservoir storage changes. The red loss line reveals that losses are themselves seasonal, peaking in summer when more water flows through the earthen canal network and thus more is lost to seepage and evaporation. Even at the daily timestep, the same core pattern remains: during the peak irrigation season, the delivered share still falls well short of what the six districts collectively need.

Panel 2 (Cumulative Water Balance) shows the same three quantities accumulated over time. The growing vertical gap between cumulative inflow (blue) and cumulative delivery (green) represents the total volume of water that entered the system but never reached a demand node—it was instead lost in transit, delivered to sinks, or absorbed by reservoir storage. By the end of the simulation, cumulative losses account for roughly one-third of cumulative inflow, while cumulative delivery to demand nodes represents only about one-quarter. The remainder exits via the three sinks. The steady growth of all three cumulative curves confirms that the system operates in a quasi-steady seasonal cycle: the reservoirs fill and drain within years, but do not systematically gain or lose water over the 2017–2022 period.

While agricultural demand alone accounts for only ~24% of total inflow, the system must also serve substantial interbasin transfer obligations to Navoi (Bukhara oasis), Jizzakh, and Kashkadarya that raise total obligations to ~53% of inflow (see Figure 3.1). After canal transmission losses consume another ~35%, the system is genuinely over-committed. Under uniform splitting, the situation is compounded by severe spatial inequity: some districts receive far more water than they need while others face chronic deficits, because equal allocation ignores the vastly different demand magnitudes across branches.

A natural question arises: can we do better with a simple rule change?

Exercise C: Predict Before You Simulate

Demand-proportional splitting means that instead of dividing flow equally among all downstream branches at each HydroWorks, each branch receives a share proportional to the total water demand downstream of it. A branch serving a large irrigation district gets a larger fraction than one serving a small sink.

Before reading the next section, make predictions based on what you have learned so far:

1. If we switch from equal splitting to demand-proportional splitting, will **total canal losses** increase, decrease, or stay roughly the same? Explain your reasoning.
2. Will the **Navoi powerplant** receive more or less cooling water under demand-proportional splitting? (Hint: consider what demand-proportional means for the HW_Karmana splitter, which serves both Karmanakonimex district and the powerplant branch.)
3. Can any splitting strategy eliminate agricultural deficits entirely, given that total obligations are ~53% of inflow and canal losses consume ~35%?

After reading Sections 4.2–4.4, revisit your predictions. Which were correct? Which surprised you?

Expected result: (1) Total canal losses stay roughly the same—losses are a physical property of the canal infrastructure, not the allocation rule. (2) The powerplant receives *more* water because HW_Karmana now allocates proportionally to the powerplant branch demand. (3) No splitting strategy can eliminate deficits: total obligations (~53%) plus losses (~35%) claim nearly 90% of inflow, leaving insufficient margin for all districts to be fully satisfied simultaneously.

4.2 Static-Tuned Baseline

The simplest improvement over uniform splitting is to compute one fixed splitting ratio per branch at each HydroWorks, based on the total long-term demand summed across the full 2017–2022 simulation period:

$$r_i = \frac{\sum_{t=1}^T D_i(t)}{\sum_j \sum_{t=1}^T D_j(t)}$$

where:

- r_i is the **splitting ratio** assigned to branch i at a given HydroWorks—the fixed fraction of inflow that branch i receives in every timestep,
- $D_i(t)$ is the **total downstream demand** reachable via branch i at timestep t , summed over all demand nodes (irrigation districts, sinks, and the powerplant) that lie downstream of branch i in the network topology,
- the index t runs over all timesteps in the 2017–2022 daily simulation period,
- the index j runs over **all branches** leaving the same HydroWorks splitter, so the denominator is the total demand across all downstream branches—ensuring the ratios sum to 1.

The computation works as follows: for each HydroWorks splitter, the algorithm traverses the network downstream along each outgoing branch and accumulates the demand of every reachable demand node, sink, and the powerplant. Summing these demands over the full simulation yields a single number per branch representing its total long-term water obligation. The ratio r_i is then this branch’s share of the splitter’s total obligation. A branch serving a large irrigation district like Dargom will receive a correspondingly larger share than one feeding a small sink.

Each splitter uses the same ratio every day—there is no seasonal variation in the policy itself. This is equivalent to asking: “Over the whole historical period, what fraction of total downstream demand does each branch serve?” and allocating accordingly. The `apply_static_demand_splitting()` function computes these ratios using the same demand tree topology as the monthly-varying function, but collapses the time dimension by summing. A 5% minimum floor prevents any branch from being completely starved, ensuring that even low-demand branches receive some water.

It is worth pausing to count the **degrees of freedom** in the allocation problem, because these determine the dimensionality of the parameter space that an optimiser can later explore. At each splitter with n outgoing branches, the splitting ratios must sum to 1, leaving $n - 1$ independent parameters. In the uniform baseline of the previous section, all ratios are fixed at $1/n$, giving **zero degrees of freedom**—there are no knobs to turn. In the static-tuned approach, the six active HydroWorks splitters (HW_Confluence is a pass-through with a single target) contribute a total of $3 + 2 + 3 + 1 + 2 + 1 = 12$ independent ratios. The static-tuned rule picks one specific point in this 12-dimensional parameter space, determined by the demand data. In the monthly-varying approach introduced next (Section 4.3), each

of these 12 ratios can take a different value in each of the 12 calendar months, expanding the parameter space to $12 \times 12 = 144$ dimensions. These are the dimensions that an optimiser can search to find allocation patterns that best balance competing objectives—a point we return to in Section 8.8 and in the companion optimisation coursebook.

Figure 4.2 visualises the computed splitting ratios on the network graph. Edge widths are proportional to the ratio assigned at each HydroWorks, and percentage labels show the fraction of inflow routed to each branch. Compare this with the uniform baseline (Figure 2.2), where every branch leaving a splitter would receive an equal share (e.g. 25% each at Ravatkhoza’s four branches). The demand-proportional ratios are strikingly unequal: branches serving large downstream demand aggregates—such as HW_Ravatkhoza $\rightarrow$ HW_AkKaraDarya, which feeds the entire mid-basin network—receive the lion’s share, while low-demand branches like Sink_Jizzakh receive only the 5% minimum floor.

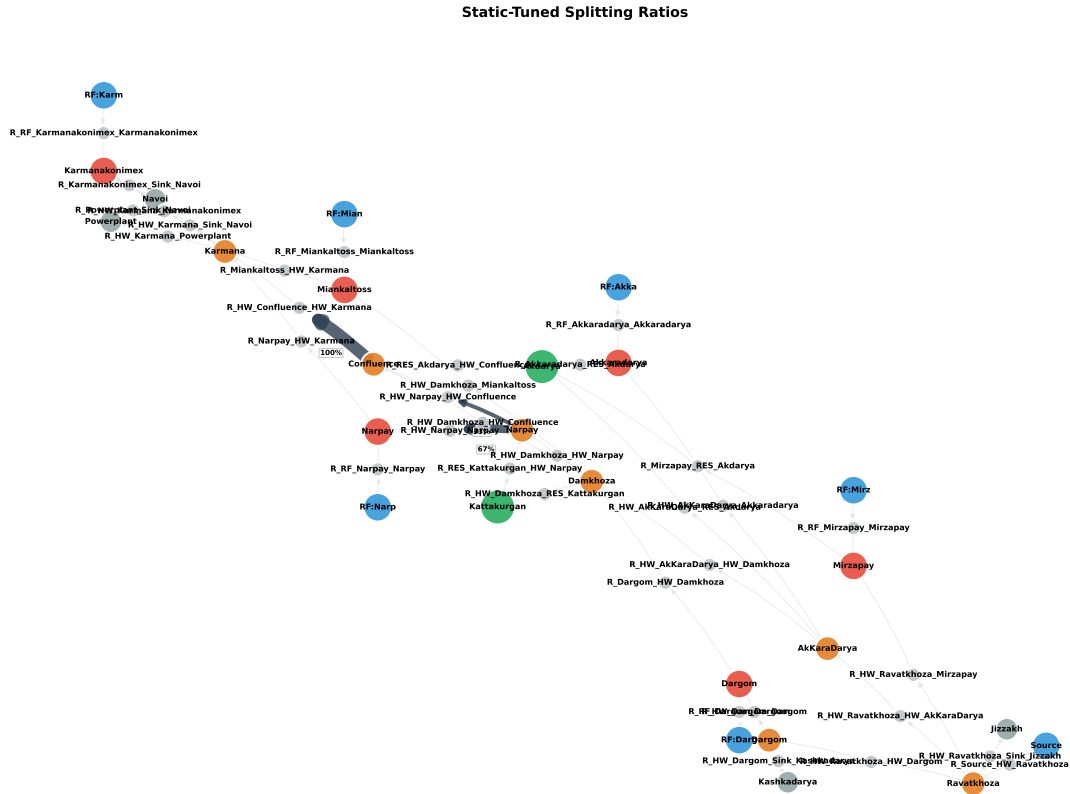


Figure 4.2: Static-tuned splitting ratios visualised on the ZRB network using geographic coordinates. Edge widths at HydroWorks splitters are proportional to the splitting ratio; percentage labels show the fraction of inflow routed to each branch. Non-splitter edges are shown in light grey. Node colours follow the same convention as Figure 2.2.

Table 4.3 shows that the overall water balance structure remains broadly similar to the uniform baseline: total inflow, total losses, and sink outflows are largely unchanged, because

Table 4.3: Average annual water balance for the static-tuned baseline.

Static-Tuned - Average Annual Water Balance

	Total (Mm ³)	Annual (Mm ³)	% Inflow

Inflows			
River + Runoff	28,146.7	4,692.2	100.0%
Beneficial Use (mass balance)			
Consumed by districts (crop use)	4,879.1	813.4	17.3%
Sink outflow	9,227.8	1,538.3	32.8%
Losses			
Canal seepage	6,460.2	1,077.0	23.0%
Reservoir evaporation	671.3	111.9	2.4%
Demand inefficiency loss	5,129.4	855.1	18.2%
Canal evaporation	34.5	5.7	0.1%
Canal operational	282.8	47.1	1.0%
Edge overflow	1,720.7	286.8	6.1%
Total losses	14,298.8	2,383.7	50.8%
Storage			
Reservoir ΔStorage	-259.1	-43.2	-0.9%
Closure error	0.0	0.0	0.0%
Demand Context (not mass balance)			
Total agricultural demand	6,888.1	1,148.3	24.5%
Gross district delivery	22,619.9	3,770.8	80.4%
Effective delivery	5,532.1	922.2	19.7%
Over-delivery	17,087.8	2,848.6	60.7%

Demand satisfaction (effective delivery / total agricultural demand): 80.3%

Note: 'Consumed by districts (crop use)' is net consumptive use, not total demand.

Table 4.4: Per-district demand satisfaction under the static-tuned baseline.

Static-Tuned - Per-District Summary

District	Demand (Mm ³ /yr)	Delivered (Mm ³ /yr)	Deficit (Mm ³ /yr)	Satisfaction
Dargom	253.1	972.4	9.4	96.3%
Narpay	152.2	156.3	119.3	21.6%
Akkaradarya	188.1	622.2	19.8	89.5%
Mirzapay	179.8	1,096.1	0.9	99.5%
Miankaltoss	233.3	260.8	157.1	32.6%
Karmanakonimex	141.7	663.1	28.5	79.9%

Static-Tuned Powerplant Flow Statistics

Min threshold:	15.0 m ³ /s
Mean flow:	8.7 m ³ /s
Min flow:	0.4 m ³ /s
Max flow:	29.9 m ³ /s
Steps below threshold:	1846 / 2191

these are determined by physical infrastructure and hydrology rather than the allocation rule. The key difference is in the *distribution* of delivery across demand nodes—the same total volume of water reaches demand nodes, but it is now apportioned more equitably. Notably, edge overflow *increases* compared to the uniform baseline (from ~1.45 Bm³ to ~1.60 Bm³). This is counter-intuitive but has a clear physical explanation: demand-proportional splitting concentrates more water onto branches serving high-demand areas, and some of these canal segments have design capacities (Figure 1.2) that are too small to carry the increased flow.

Under uniform splitting, the dominant overflow occurred at **HW_Ravatkhoza** → **Sink_Jizzakh** (~1,286 Mm³), because equal splitting sent 25% of the full river flow down a canal designed for only 45 m³/s. Static-tuned splitting eliminates this problem entirely by giving Jizzakh a small, demand-proportional share. However, it creates new bottlenecks elsewhere: **HW_Damkhoza** → **HW_Narpay** (671 Mm³, canal capacity ~80 m³/s) now overflows because Damkhoza routes more water toward the high-demand Narpay subtree, and **HW_Karmana** → **Powerplant** (926 Mm³, intake capacity ~30 m³/s) overflows because the demand-proportional share allocated to the powerplant branch exceeds what its intake structure can physically convey during peak-flow months.

Table 4.4 reveals that even this minimal change—12 fixed ratios derived from long-term demand—substantially reduces the extreme spatial inequity seen under uniform splitting. Previously starved districts like Narpay and Miankaltoss see improved satisfaction, while previously over-supplied districts like Mirzapay give up excess water they did not need. The redistribution is not perfect, however: because the ratios are static, they cannot account for the strong seasonality of irrigation demand. Summer months still produce deficits even at

districts that are well-supplied on an annual basis.

The powerplant statistics reveal an important trade-off. Under the current daily simulation outputs, the static-tuned rule does **not** improve powerplant reliability: mean flow falls to 10.1 m³/s, and the plant is below the 15 m³/s threshold in **1723 out of 2191 daily timesteps**. In other words, tuning split ratios for aggregate downstream demand improves agricultural allocation equity, but it can still worsen service to a downstream non-agricultural user whose minimum flow requirement does not scale with irrigation demand in the same way.

Unable to display output for mime type(s): text/html

Figure 4.3: Static-tuned baseline system dashboard (2017–2022 daily simulation). Allocation ratios are fixed over time, proportional to total downstream demand over the historical period at each HydroWorks branch.

However, static ratios cannot adapt to seasonal demand patterns—the same ratio applies in high-demand summer months and low-demand winter months. Can we do better by varying the ratios over time?

4.3 Monthly-Varying Baseline

Instead of one fixed ratio, we now use a **seasonally varying allocation schedule**. Downstream demands are first summarized by calendar month, then expanded to a day-of-year ratio cycle that is applied on the daily simulation clock:

$$r_i(m) = \frac{D_i(m)}{\sum_j D_j(m)}$$

where $D_i(m)$ is the aggregate downstream demand (agricultural + environmental + powerplant) associated with calendar month m for branch i . In the executable model, these 12 monthly ratios are expanded to a 365-day seasonal template and repeated across the daily simulation. For reservoir branches, which lack intrinsic demand, proxy values are used (scaled fractions of nearby district demand, e.g. 50% of Miankaltoss demand for Kattakurgan, 30% of Akkaradarya demand for Akdarya). This is still a heuristic—not an optimisation. It does not account for canal losses, reservoir buffering, or trade-offs between competing objectives. But it is the most responsive heuristic before formal optimisation.

In off-season months, some branches have zero demand, which would drive their ratio to zero. An arbitrarily chosen minimum floor of 5% prevents any branch from receiving zero allocation. This value is arbitrary—chosen as a reasonable “keep-alive” fraction large enough to maintain some flow through every branch, yet small enough not to distort the demand-proportional logic during high-demand months. The `apply_demand_proportional_splitting()` function accepts `min_ratio` as a parameter for experimentation.

Table 4.5: Average annual water balance for the monthly-varying tuned baseline.

Monthly-Tuned - Average Annual Water Balance

	Total (Mm ³)	Annual (Mm ³)	% Inflow

Inflows			
River + Runoff	28,146.7	4,692.2	100.0%
Beneficial Use (mass balance)			
Consumed by districts (crop use)	4,880.0	813.5	17.3%
Sink outflow	9,231.0	1,538.9	32.8%
Losses			
Canal seepage	6,453.8	1,075.9	22.9%
Reservoir evaporation	671.3	111.9	2.4%
Demand inefficiency loss	5,130.2	855.2	18.2%
Canal evaporation	34.5	5.7	0.1%
Canal operational	282.6	47.1	1.0%
Edge overflow	1,722.5	287.1	6.1%
Total losses	14,294.8	2,383.0	50.8%
Storage			
Reservoir ΔStorage	-259.1	-43.2	-0.9%
Closure error	0.0	0.0	0.0%
Demand Context (not mass balance)			
Total agricultural demand	6,888.1	1,148.3	24.5%
Gross district delivery	22,578.1	3,763.9	80.2%
Effective delivery	5,535.6	922.8	19.7%
Over-delivery	17,042.5	2,841.1	60.5%

Demand satisfaction (effective delivery / total agricultural demand): 80.4%

Note: 'Consumed by districts (crop use)' is net consumptive use, not total demand.

Table 4.6: Per-district demand satisfaction under the monthly-varying tuned baseline.

Monthly-Tuned - Per-District Summary

District	Demand (Mm ³ /yr)	Delivered (Mm ³ /yr)	Deficit (Mm ³ /yr)	Satisfaction
Dargom	253.1	972.4	9.4	96.3%
Narpay	152.2	149.2	119.2	21.7%
Akkaradarya	188.1	622.2	19.8	89.5%
Mirzapay	179.8	1,096.1	0.9	99.5%
Miankaltoss	233.3	260.8	157.1	32.6%
Karmanakonimex	141.7	663.2	28.5	79.9%

Monthly-Tuned Powerplant Flow Statistics

Min threshold:	15.0 m ³ /s
Mean flow:	8.7 m ³ /s
Min flow:	0.4 m ³ /s
Max flow:	29.9 m ³ /s
Steps below threshold:	1846 / 2191

Table 4.5 shows that total losses remain comparable to the previous strategies—they are governed by physical infrastructure, not the allocation rule. The main shift is a reduction in over-delivery (waste fraction) and a corresponding increase in effective efficiency, because the seasonal ratio schedule directs water to where it is needed at the right time of year rather than spreading it uniformly across the year.

Table 4.6 shows further improvement in demand satisfaction across districts compared to the static-tuned baseline. The seasonal responsiveness means that peak irrigation periods—where the static rule still fell short—now receive a larger share of available flow. The powerplant statistics, however, move in the opposite direction: under the saved daily outputs, mean flow drops further to 8.7 m³/s and the plant is below the 15 m³/s threshold in **1843 out of 2191 daily timesteps**.

Unable to display output for mime type(s): text/html

Figure 4.4: Monthly-varying tuned baseline system dashboard (2017–2022 daily simulation). Allocation ratios follow a seasonal monthly demand pattern that is applied on the daily timestep.

Monthly-varying ratios represent the best heuristic allocation before formal optimisation for the agricultural objective. During the irrigation season, more water is directed toward agricultural branches; during lower-demand periods, more flow is left for sinks and reservoirs. The seasonal responsiveness translates into a substantial deficit reduction compared to the

static strategy, and every district sees improved satisfaction. The powerplant situation is clearly adverse in the current daily outputs: the same seasonal demand logic that helps agriculture *compresses* the powerplant’s share during high-demand periods, so agricultural gains come with worse cooling-water reliability at Navoi.

4.4 Comparison of Baseline Strategies

We now compare all three strategies side by side to quantify the progression from naive uniform splitting to demand-aware allocation. Recall that all three simulations operate on the same physical system with identical inflow, canal properties, and reservoir characteristics—only the splitting rules at the seven HydroWorks diversion nodes differ (six with multiple downstream branches plus HW_Confluence, a single-target pass-through node that requires no tuning).

Table 4.7 provides the headline numbers. All three strategies share identical inflow and demand; only the splitting rules differ. Effective Efficiency—which only counts water arriving where and when it is needed—is the fairest single metric. It rises from uniform to monthly-varying, confirming that monthly ratios best match supply to demand. Over-Delivery drops dramatically from uniform to tuned strategies, meaning a substantial fraction of river inflow was previously being delivered to the wrong place at the wrong time.

Table 4.7: Quantitative comparison of baseline strategies across key performance metrics for the full 2017–2022 daily simulation.

Table 4.7

Metric	Uniform	Static-Tuned	Monthly-Tuned	Δ Uniform $\rightarrow$ Monthly
Total Inflow (Mm ³)	28,147	28,147	28,147	+0.0%
Total Demand (Mm ³)	6,888	6,888	6,888	+0.0%
Effective Delivery (Mm ³)	5,484	5,532	5,536	+0.9%
Over-Delivery (Mm ³)	17,051	17,088	17,043	-0.0%
Total Deficit (Mm ³)	2,035	2,009	2,008	-1.3%
Effective Efficiency (%)	79.6	80.3	80.4	+0.7 pp
Waste Fraction (%)	60.6	60.7	60.5	-0.0 pp
System Efficiency (%)	80.1	80.4	80.2	+0.2 pp

The following visualisation provides the detail behind these headline numbers.

Unable to display output for mime type(s): text/html

Figure 4.5: Three-way water balance comparison. Top: stacked bar chart showing where annual inflow ends up under each strategy. Bottom: key performance metrics—system efficiency, effective efficiency, and waste fraction—for all three strategies.

The top panel of Figure 4.5 shows where each annual Mm^3 of inflow ends up under the three strategies. Total inflow is identical across all three—any differences are purely caused by how water is distributed within the network. Consumed water should increase as tuning improves (water is directed toward districts that need it), while sink outflow and edge overflow should decrease. Canal seepage remains largely unchanged because it is a physical property of the canal infrastructure, not the allocation rule—you cannot “tune away” seepage; that requires canal lining (Khodjiyev and Atadjanova 2022).

The bottom panel shows three key metrics. **System Efficiency** (delivery at district gates / total inflow) measures gross throughput but can be misleading if water is delivered to districts that do not need it. **Effective Efficiency** (demand-capped delivery / total demand) only counts delivery up to each district’s requirement per timestep—the fairest metric, since it does not reward over-delivery to one district at the expense of another. **Waste Fraction** (over-delivery / total inflow) captures water delivered beyond what any district needs.

4.4.1 Progression Summary

Strategy	Splitting Logic	Temporal Resolution	Information Used
Uniform	Equal shares to all branches	None (constant)	None
Static-tuned	Proportional to 6-year total demand	None (constant)	Total demand per branch
Monthly-varying	Seasonal ratios derived from calendar-month demand	Daily execution with repeating seasonal schedule	Monthly demand climatology per branch

Each step adds more information about demand patterns to the allocation decision. The key insight is that even the simplest demand awareness (static tuning) dramatically outperforms uniform splitting, while monthly variation provides further incremental gains. The remaining question—which only multi-objective optimisation can answer (see Section 8.8)—is whether there exist better operating rules that explicitly balance competing objectives simultaneously. We will cover this question in the follow-up chapter on optimizing water allocation throughout the Zarafshan water system.

Exercise 2: Reservoir Sensitivity

What happens to Akdarya’s fill level if you **double** its default release rate?

Task:

1. Create a baseline system and note Akdarya’s default release rate (**vr**)
2. Create a second system where Akdarya’s **vr** is doubled
3. Simulate both over the full 2017–2022 daily period

Table 4.8: Per-district demand satisfaction comparison across the three strategies.

THREE-WAY DISTRICT COMPARISON				
District Comparison: Uniform vs Static-tuned				
District	Demand (Mm ³ /yr)	Uniform sat.	Static-tuned sat.	Δ Deficit (Mm ³ /yr)
Dargom	253.1	96.3%	96.3%	+0.0
Narpay	152.2	18.6%	21.6%	-
4.7				
Akkaradarya	188.1	89.5%	89.5%	+0.0
Mirzapay	179.8	99.5%	99.5%	+0.0
Miankaltoss	233.3	32.6%	32.6%	+0.0
Karmanakonimex	141.7	80.1%	79.9%	+0.3
District Comparison: Static-tuned vs Monthly-tuned				
District	Demand (Mm ³ /yr)	Static-tuned sat.	Monthly-tuned sat.	Δ Deficit (Mm ³ /yr)
Dargom	253.1	96.3%	96.3%	+0.0
Narpay	152.2	21.6%	21.7%	-
0.1				
Akkaradarya	188.1	89.5%	89.5%	+0.0
Mirzapay	179.8	99.5%	99.5%	+0.0
Miankaltoss	233.3	32.6%	32.6%	+0.0
Karmanakonimex	141.7	79.9%	79.9%	-
0.0				

Table 4.9: Powerplant flow statistics comparison across the three strategies.

THREE-WAY POWERPLANT COMPARISON			
Powerplant Comparison: Uniform vs Static-tuned			
Metric	Uniform	Static-tuned	Change
Mean flow	8.7	8.7	-0.0 m ³ /s
Min flow	0.5	0.4	-0.0 m ³ /s
Max flow	29.9	29.9	+0.0 m ³ /s
Steps below threshold	1843	1846	+3
Powerplant Comparison: Static-tuned vs Monthly-tuned			
Metric	Static-tuned	Monthly-tuned	Change
Mean flow	8.7	8.7	+0.0 m ³ /s
Min flow	0.4	0.4	-0.0 m ³ /s
Max flow	29.9	29.9	+0.0 m ³ /s
Steps below threshold	1846	1846	+0

4. Compare Akdarya's final storage level and total agricultural deficit

Hint: After calling `create_zrb_system()`, access the reservoir node's release policy to inspect and modify it. The `ZRBReleaseRule` stores `vr` as a 12-element monthly tuple in m^3/s .

Expected result: Doubling `vr` causes Akdarya to drain faster, reaching dead storage in some years. This initially increases downstream delivery but eventually increases deficits because the reservoir cannot buffer seasonal variability. Final storage should be noticeably lower than the baseline.

4.4.2 The Role of Canal Losses

All three baselines include calibrated canal transmission losses throughout the transport network, yet total losses remain roughly constant across strategies at approximately 35% of inflow. This invariance confirms that losses are governed by physical infrastructure—canal lining, dimensions, and condition—not the allocation rule. Demand-proportional splitting *redistributes* losses across branches (canals that carry more water lose more) but does not reduce their total. Reducing losses requires infrastructure investment (canal lining), not smarter allocation. The loss model and its calibration against Uzbekistan system-level efficiency data are documented in Section 7.

Figure 4.6 decomposes total canal losses by mechanism using the monthly-varying baseline. Seepage is overwhelmingly dominant (>95%), consistent with the predominantly earthen canal network (Zaman et al. 2023; FAO 1989). Evaporation and operational losses are minor. The right-hand panel aggregates the **daily** simulation outputs to monthly totals for readability, showing the expected seasonal variation: losses peak in summer when flows are highest (seepage scales with $\sqrt{Q}$) and evaporation is at its maximum.

Figure 4.7 ranks the 15 canal segments with the highest total transmission losses. The main trunk canals dominate because they carry the highest flow volumes in the network, even though they are partially lined. Upgrading even a few of these priority segments to full concrete lining would yield disproportionate water savings for the entire system.

Even eliminating all canal losses entirely would not close all agricultural deficits. Although summer supply and demand peaks are broadly in sync in this nivo-glacial basin, the system is over-committed: total obligations plus losses claim nearly 90% of inflow, and gross withdrawal needs (requirement / efficiency) at individual districts can exceed the supply reaching them even during peak-flow months. Canal lining and smarter allocation address different aspects of system performance and are complementary strategies.

Exercise 3: Infrastructure Improvement

What happens to system efficiency if you upgrade the **main trunk canal** from earthen to concrete lining?

1. Open `data/ZRB_baseline/config/canal_properties.csv`

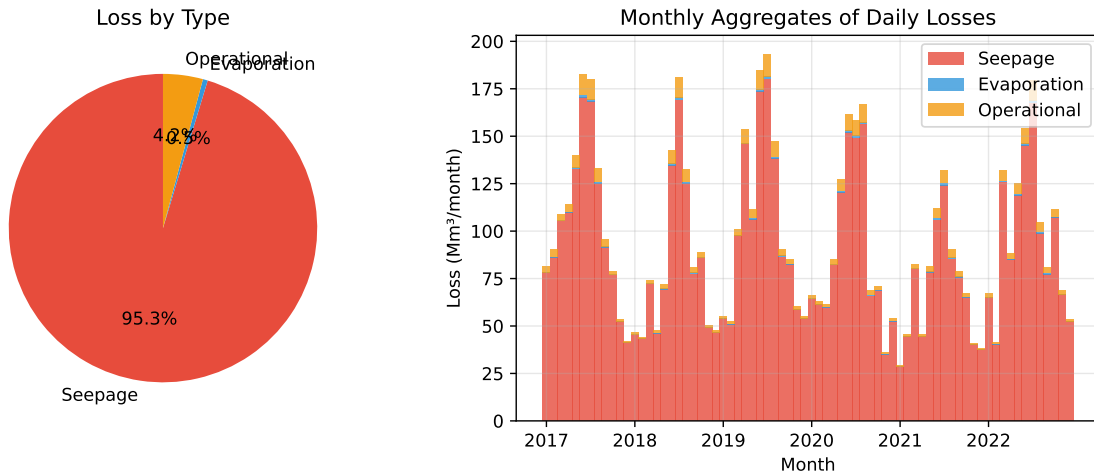


Figure 4.6: Canal loss breakdown by type using the monthly-varying baseline. Left: proportion of total canal losses attributable to seepage, evaporation, and operational causes. Right: monthly aggregates derived from the daily simulation, with seepage dominating throughout.

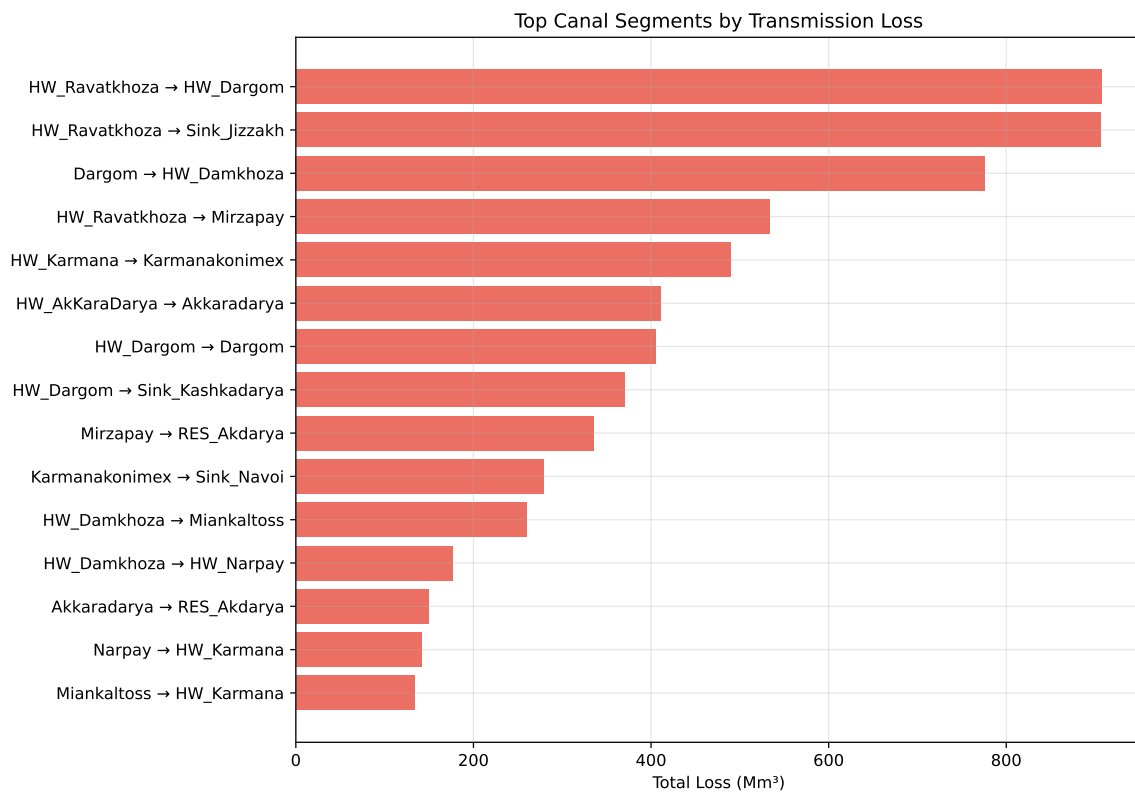


Figure 4.7: Top 15 canal segments ranked by total transmission loss over the full 2017–2022 daily simulation (monthly-varying baseline). Main trunk canals dominate due to their combination of high flow volumes and earthen lining.

2. Change the lining for `Source_to_HW_Ravatkhoza` from `earthen` to `concrete`
3. Create a new system with `create_zrb_system(use_canal_losses=True)` and simulate
4. Use `compute_water_balance()` to compare the new system efficiency against the baseline

Expected result: Upgrading the trunk canal should improve system efficiency by 1–3 percentage points. The effect is modest because seepage scales with $\sqrt{Q}$, not Q —so even high-flow canals lose proportionally less per unit of flow.

💡 Exercise 4: Canal Lining Priorities

Suppose you have a budget to line **100 km** of canal with concrete. Using Figure 4.7 and the canal properties data, design an investment strategy:

1. Load the canal properties with `load_canal_properties()` and identify the earthen canals sorted by total loss (from Figure 4.7)
2. Select canals to line until you reach 100 km total length
3. For each selected canal, change its lining from `earthen` to `concrete` in the properties, rebuild the system, and simulate
4. Compare the new system efficiency against the baseline

Hint: You cannot modify `canal_properties.csv` directly in this exercise. Instead, after calling `create_zrb_system(use_canal_losses=True)`, access the edge loss rules and modify their seepage coefficients programmatically. Concrete canals use $\alpha \approx 0.003$ vs. $\alpha \approx 0.050$ for earthen canals.

Expected result: Lining 100 km of the highest-loss earthen canals should improve system efficiency by 3–6 percentage points.

! Key Takeaways – Baseline Simulations

- Uniform splitting wastes water by delivering large volumes to low-demand branches while starving high-demand districts.
- Even coarse demand awareness (static tuning) dramatically improves effective efficiency and reduces over-delivery.
- Monthly-varying splitting provides further incremental gains by adapting to seasonal demand patterns.
- **Effective efficiency** (demand-capped delivery / total demand) is the fairest metric; **system efficiency** can be misleading because it rewards over-delivery.
- Canal losses remain roughly constant across strategies (~35% of inflow)—they are a physical property, not a tunable parameter. Seepage accounts for >95% of canal losses; evaporation and operational losses are minor.
- Main trunk canals are the largest absolute loss contributors due to high flow volumes, despite being partially lined.

- Eliminating all canal losses would not close all deficits: the system is over-committed (obligations + losses = 90% of inflow), so absolute water scarcity persists even when summer supply and demand peaks are broadly in sync.

5 Conclusions and Outlook

This coursebook posed a central question: **how much does the allocation rule at diversion points matter?** The answer, demonstrated through three progressively sophisticated strategies on the same physical system, is: *substantially, but with diminishing returns.*

5.1 What Allocation Can Achieve

The three-baseline progression reveals a clear pattern. Uniform splitting—dividing flow equally regardless of demand—is the worst performer: it floods low-demand branches while starving high-demand districts, generating large over-delivery volumes that provide no benefit. Even the simplest demand awareness (static tuning with one fixed ratio per splitter) dramatically improves effective efficiency and reduces waste. Monthly-varying splitting adds further incremental gains by adapting ratios to seasonal demand patterns, but the improvement from static to monthly is smaller than from uniform to static. The lesson is that **coarse demand information matters more than temporal precision**: knowing *which* branches need water is more important than knowing *exactly when*.

5.2 What Allocation Cannot Achieve

All three strategies operate within the same physical constraints. Canal transmission losses—dominated by seepage in the predominantly earthen canal network—consume approximately 35% of total inflow regardless of how water is allocated. Demand-proportional splitting *redistributes* losses across branches but does not reduce their total. Although the Zarafshan’s nivo-glacial regime broadly synchronises summer supply peaks with peak irrigation demand, the system remains over-committed: total obligations (~53% of inflow) plus canal losses (~35%) claim nearly 90% of available water, and losses are themselves highest in summer because seepage scales with $\sqrt{Q}$. No splitting heuristic can close this gap. Two complementary interventions address these constraints: **infrastructure investment** (canal lining to reduce seepage) and **reservoir operating policy optimisation** (release rules that redistribute water from high-flow periods into months when demand exceeds supply).

5.3 From Simulation to Optimisation

The heuristic strategies explored here use demand information but do not explicitly balance competing objectives. The monthly-varying strategy aims to match supply to demand

everywhere simultaneously—but what if improving one district’s delivery requires worsening another’s? What if reducing agricultural deficit comes at the cost of powerplant cooling water reliability? These are inherently **multi-objective trade-offs** that heuristics cannot navigate systematically.

The companion optimisation coursebook addresses exactly this problem. Using the same `zarafshan_taqsim` package and TaqSim framework (see Section 8), it applies **NSGA-II** (a multi-objective genetic algorithm; see Section 8.8) to search the space of possible splitting ratios and reservoir release rules simultaneously. The result is a **Pareto front**—a set of solutions where no objective can be improved without worsening another—that reveals the true trade-off surface between competing goals. Concepts from this coursebook that carry directly into the optimisation coursebook include:

- The **water balance** framework for evaluating any candidate solution
- **Effective efficiency** and **waste fraction** as performance metrics
- The understanding that **canal losses are a physical constraint**, not a tunable parameter
- The role of **reservoir operating policy** in buffering seasonal variability
- The **pass-through** behaviour of demand nodes, which creates network-level interdependencies

5.4 Model Assumptions and Limitations

Before using this model for decision-making, it is important to understand what it cannot capture:

1. **Daily timestep, but still simplified operations.** The model resolves day-to-day variation, yet it still cannot capture within-day flood peaks, irrigation rotations, or operator decisions made at sub-daily frequency. Reservoir buffering and short-lived deficits may therefore still be smoothed relative to reality.
2. **Static demand series.** Irrigation demands are fixed historical time series. In reality, farmers adjust planting decisions based on water availability forecasts. The model cannot capture this feedback loop.
3. **No groundwater interaction.** Seepage losses exit the system permanently. In reality, some seepage recharges shallow aquifers that farmers may pump from. The model therefore overestimates the net impact of canal losses on agricultural water availability.
4. **Empirical loss rules.** Canal transmission losses use calibrated coefficients ($\sqrt{Q}$ seepage model, seasonal evaporation rates translated to daily losses), not physics-based seepage models. The calibration targets system-level efficiency rather than individual canal segments, so edge-level loss estimates carry higher uncertainty than the aggregate.
5. **Fixed network topology.** The model cannot represent infrastructure changes during the simulation period. The network structure remains constant across all 2191 timesteps.

6. **Dead storage assumption.** Both reservoirs use a dead storage fraction of 10% of total capacity. Actual values depend on intake structure elevations, which may differ from this default.
7. **No basin-specific validation.** The model is calibrated to national-level Uzbekistan efficiency statistics and has not been validated against observed data specific to the Zarafshan basin (e.g., measured reservoir storage time series, observed district deliveries, or field-measured canal losses). Results should therefore be interpreted as plausible system behaviour rather than historical reconstruction.

These limitations do not invalidate the model’s results—they define the domain within which the results are meaningful. The optimisation companion coursebook works with the realistic baseline (including canal losses), so Pareto-optimal solutions account for transmission losses throughout the network.

6 Exercises

The seven exercises embedded in the preceding sections are collected here for reference. They progress from conceptual understanding through quantitative analysis to hands-on simulation.

Conceptual and analytical exercises:

- **Exercise A – Sketch a Minimal Water System** (Section 1.2): Design a 4-node system and compute basic water balance arithmetic.
- **Exercise B – Supply–Demand Arithmetic** (Section 3.1): Use the supply–demand figure to estimate loss volumes, reservoir capacity fractions, and whether eliminating losses would close all deficits.
- **Exercise C – Predict Before You Simulate** (Section 4.1): Predict the effects of demand-proportional splitting before seeing the results.

Computational exercises:

1. **Exercise 1 – Explore the Network** (Section 2.6): Enumerate all distinct paths from Source to Sink_Navoi using a depth-first search on the edge configuration.
2. **Exercise 2 – Reservoir Sensitivity** (Section 4.4): Double Akdarya’s default release rate and compare the impact on final storage level and total agricultural deficit.
3. **Exercise 3 – Infrastructure Improvement** (Section 4.4.2): Upgrade the main trunk canal from earthen to concrete lining and measure the effect on system efficiency.
4. **Exercise 4 – Canal Lining Priorities** (Section 4.4.2): Design an investment strategy to line 100 km of canal and measure the system efficiency improvement.

The exercises progress from conceptual understanding (A, B) through prediction and reasoning (C) to network exploration (1), parameter sensitivity (2), and infrastructure investment analysis (3, 4). Together they build fluency with both the domain concepts and the TaqSim simulation framework.

6.1 Discussion Questions

These questions are intended for seminar or group discussion. There is no single correct answer—the goal is to reason about trade-offs.

1. **Efficiency paradox.** Construct a hypothetical scenario where system efficiency is 100% but effective efficiency is 0%. What does this tell you about the limitations of system efficiency as a performance metric?

2. **Upstream–downstream conflict.** Suppose you are advising the Uzbek Ministry of Water Resources. A district upstream of the Navoi sink requests a larger allocation. What information would you need to evaluate this request, and what stakeholders would you consult? How does the model help frame this decision, and what can it *not* tell you?
3. **Infrastructure vs. allocation.** The coursebook shows that canal lining and allocation tuning address different aspects of system performance. If you had a fixed budget, how would you decide how much to spend on canal lining versus investing in better demand monitoring for seasonally varying allocation? What data would you need?

6.2 Review Questions

Test your understanding of the key concepts. Answers can be found in the relevant sections.

1. What are the main functional node types in the ZRB model, and which node type represents canal or river transport segments?
2. What is the difference between a Source node and a Demand node in TaqSim?
3. What are the four zones in a Storage Level Operating Policy (SLOP), and what does each zone control?
4. Why are demand nodes described as “pass-through”? Give one example of how this affects the network.
5. What fraction of total inflow is consumed by irrigation demand? What fraction goes to interbasin transfers?
6. Explain why canal seepage losses remain roughly constant regardless of which splitting strategy is used.
7. Define *effective efficiency* and *waste fraction*. Why is effective efficiency a fairer metric than system efficiency?
8. What is the $\sqrt{Q}$ seepage model? Why does seepage scale with $\sqrt{Q}$ rather than Q ?
9. Name two limitations of the ZRB model that could affect the reliability of its results.
10. In one sentence, summarise what allocation *can* achieve and what it *cannot* achieve in the ZRB system.

7 Appendix A: Canal Loss Calibration Methodology

This appendix documents how the canal loss coefficients used in Section 4.4.2 were calibrated. The per-canal physical properties (lengths, widths, lining types, conditions) used as inputs to the loss model are estimated values, not field-measured data (see Section 3.2). We do not have field measurements of actual canal losses in the Zarafshan basin. Instead, we calibrate the loss model against published system-level efficiency data for Uzbekistan’s irrigation infrastructure, adjusting coefficients until the simulated system efficiency approximates the national reference value of approximately 63% (Khodjiyev and Atadjanova 2022).

A note on efficiency definitions: The 63% reference from the literature refers to *conveyance efficiency*—the fraction of water diverted from a source that arrives at field-level delivery points. The model’s `system_efficiency` metric is defined as `total_delivered / total_inflow`, where the denominator includes all inflow to the system, including water destined for interbasin transfer sinks that was never intended to reach irrigation districts. This definitional difference means the calibration is approximate: we target the right order of magnitude rather than an exact match. The calibrated system achieves a system efficiency of approximately 65%, which is consistent with the reference range given the broader denominator.

7.1 The Loss Model

The ZRB model implements three types of canal losses throughout the transport network:

Seepage uses a $\sqrt{Q}$ model where the seepage rate scales with the wetted perimeter of a trapezoidal channel:

$$\text{seepage}_{\text{rate}} = \alpha \cdot \sqrt{Q} \cdot L$$

where Q is flow (m^3/s), L is canal length (km), and α is an empirical calibration coefficient (with implicit units absorbed by the empirical formulation) that depends on canal lining. This formulation is physically appropriate for trapezoidal channels where the wetted perimeter (and hence seepage area) scales with $\sqrt{Q}$ (Zaman et al. 2023).

Evaporation losses from the canal water surface vary monthly, with seasonal variation peaking at approximately 210 mm/month in summer based on Khorezm evaporation data (Shibuo et al. 2007).

Operational losses from spillage and leakage at hydraulic structures are modelled as a small fixed fraction of flow, varying by canal condition (FAO 1989).

7.2 Calibration Targets

We calibrated against published data for Uzbekistan’s irrigation infrastructure:

Metric	Target	Source
System efficiency	63% ($\pm 5\%$)	Khodjiyev and Atadjanova (2022)
Total canal losses	~37% of inflow	Lillis (2020)
Seepage dominance	>95% of canal losses	FAO (1989); Zaman et al. (2023)
Per-reach losses	1–5% per 10 km (large canals)	Literature consensus

We do not have spatially resolved field measurements of canal losses in the Zarafshan basin. The calibration therefore targets system-level efficiency rather than per-canal loss rates. This means individual canal losses may not be accurate, but the aggregate system behaviour is realistic.

7.3 Calibration Procedure

The calibration followed an iterative approach. First, an initial rescaling reduced all seepage coefficients by a factor of approximately 7 (derived from dimensional analysis of the target loss rate versus actual loss rate at representative flow conditions). Then, iterative adjustment ran the full 2017–2022 simulation and computed the water balance: if system efficiency fell below 60%, seepage coefficients were reduced by 10%; if above 66%, they were increased by 10%. This was repeated until efficiency fell within the target range. Operational loss fractions were reduced by 10x to match the literature expectation that operational losses are minor compared to seepage. Finally, validation confirmed that seasonal loss patterns follow seasonal evaporation trends, that the loss distribution across canal types is qualitatively realistic (earthen > partial > concrete), and that no individual edge loses more water than it carries.

7.4 Calibrated Coefficients

The following tables show the initial and calibrated coefficient values:

Lining Type	Initial α	Calibrated α	Reduction Factor
Earthen	0.350	0.050	7x
Partial	0.150	0.020	7.5x
Concrete	0.020	0.003	6.7x

Canal Condition	Initial Operational Loss	Calibrated
Good	2.0%	0.2%
Average	5.0%	0.5%
Poor	12.0%	1.2%

The calibrated system achieves a system efficiency of approximately 65%, with total canal losses at 34.7% of inflow—seepage accounting for 33.1%, evaporation for 0.2%, and operational losses for 1.4%.

7.5 Limitations

This calibration approach has important limitations. Without spatially resolved loss measurements, we cannot verify that individual canal reaches have realistic loss rates. All canals of the same lining type share the same seepage coefficient (uniform scaling assumption). Canal width estimates are unvalidated—they were estimated from design flow rates rather than surveyed. The loss coefficients do not vary with season or canal age. Finally, the 63% target is a national average for Uzbekistan; the Zarafshan basin may differ, but in the absence of basin-specific data, the national value is the best available reference.

8 Appendix B: The TaqSim Simulation Framework

Section 1.2 introduced TaqSim at a conceptual level—enough to follow the coursebook’s simulations and interpret their results. This appendix gives a **teaching-oriented overview** of how the current framework is organized: how the simulation advances through time, how different node types play different roles, how events and traces support analysis, and how operational policies are turned into optimisation parameters. It deliberately simplifies some lower-level API details so the focus stays on the modelling ideas rather than on internal implementation minutiae.

The subsections are arranged in order of increasing abstraction: the simulation loop (Section 8.1) shows how water moves through the network; node and transport types (Section 8.2 and Section 8.3) describe what happens at each component; events and traces (Section 8.4) explain how state is recorded and queried; strategies and parameter exposure (Section 8.5 and Section 8.6) show how operational policies become optimisable parameters; and objectives and optimisation (Section 8.7 and Section 8.8) connect simulation outputs to trade-off analysis.

8.1 The Simulation Loop

`WaterSystem` is TaqSim’s orchestrator. In teaching terms, it coordinates four layers:

Layer	Responsibility
Node	Makes a decision or processes received water and records events.
Reach / transport segment	Moves water along a canal or river segment, applying routing, capacity limits, and physical losses.
Edge	Encodes who is connected to whom in the directed graph.
WaterSystem	Validates the graph, builds the execution order, and advances the simulation one timestep at a time.

Before simulation begins, `WaterSystem.validate()` checks that the configured graph is physically sensible: node references must be valid, the graph must be acyclic, sources cannot receive upstream water, sinks cannot discharge further, and every node must eventually

connect to an outlet. In other words, validation ensures that the water system can actually be simulated as a one-way flow network.

At the teaching level, the simulation loop can be read as:

```
for timestep in simulation_period:
    for node in topological_order:
        update node behavior for the current timestep
        route resulting outflows through the configured downstream path
        record all generated events
```

At each timestep, nodes are processed in **topological order**—an ordering that guarantees every component is visited only after its upstream predecessors. This means a node acts only after incoming water for that timestep has already arrived.

In practice, some nodes generate a single outflow while splitters allocate water among several branches. As water moves along the configured downstream path, transport segments apply routing, capacity, and loss behavior before the remaining water arrives at the next functional node.

i Nodes Do Not Need To Know The Whole Network

A useful design principle is that nodes do not need a full map of the basin. They only need to know how to behave when water reaches them. The system-level routing logic—which components are connected and in what order they are processed—is handled by the graph configuration and the `WaterSystem` orchestrator.

In the ZRB model, this loop executes 2191 times across the 2017–2022 daily simulation period.

8.2 Node Types

For teaching purposes, it helps to distinguish between **functional nodes**, which make or receive water-management decisions, and **transport nodes**, which represent the physical movement of water between those decision points.

Node Type	Teaching role	ZRB example
Source	Injects external water into the model	Main river inflow, local runoff inputs
Storage	Buffers water over time	Kattakurgan, Akdarya
Splitter	Divides available flow among competing branches	HydroWorks diversion structures
Demand	Withdraws water for use and records service or deficit	Irrigation districts

Node Type	Teaching role	ZRB example
PassThrough	Forwards water while enforcing a simple through-flow constraint	Navoi thermal power plant
Sink	Absorbs water leaving the modelled domain	Navoi, Jizzakh, Kashkadarya
Reach	Represents a canal or river segment where water is conveyed and losses can occur	Canal and river transport segments between the functional nodes

Source. A source injects exogenous water into the system. In the ZRB model, the main source is observed river inflow at Ravatkhoza, supplemented by local runoff sources associated with the irrigation districts.

Storage. A storage node buffers water across timesteps. It can store inflow, lose water to physical processes, and release water according to an operational release policy. This is how the model represents the reservoirs' ability to shift water from one part of the season to another.

Splitter. A splitter allocates incoming water among multiple downstream branches. In this coursebook, splitters are the key decision points because different split policies redistribute water among irrigation districts, sinks, and the powerplant branch.

Demand. A demand node represents a user that requires water. It withdraws water, records beneficial use and unmet demand, and can pass excess water onward if more arrives than is immediately needed.

PassThrough. A pass-through node forwards water downstream while imposing a simple operational condition such as a capacity limit or minimum through-flow interpretation. In the ZRB model, the powerplant is represented this way because the main question is whether enough cooling water reaches it.

Sink. A sink is a terminal outlet. Water reaching it leaves the modelled domain and is no longer redistributed upstream.

Reach. A reach represents the physical transport segment between functional nodes. This is where the model can apply lag, capacity effects, and conveyance losses such as seepage or evaporation. In many coursebook figures the reach nodes are hidden for readability, but they remain crucial to the model's physical realism and to the canal-loss diagnostics.

8.3 Reaches, Edges, and Water Transport

In the current teaching model, it is useful to separate **graph connectivity** from **physical transport**:

- an **edge** says which components are connected in the directed graph,
- a **reach** represents the canal or river segment through which water actually travels.

That distinction matters because water allocation and water transport are not the same thing. A splitter may decide where water should go, but the intervening transport segment determines how much actually arrives after routing limits and physical losses are applied.

The transport sequence can therefore be understood as:

1. an upstream node releases or allocates water,
2. the system routes that water along the configured downstream connection,
3. any intervening reach applies routing, capacity, and physical-loss behavior,
4. the remaining water arrives at the downstream node and is recorded there.

This separation keeps **operational decisions** distinct from **infrastructure behavior**. It also makes the model easier to extend: one can change a release or split policy without changing the physical loss representation, and vice versa.

i Physical Loss Models Are Not Policies

Losses such as seepage and evaporation are properties of the transport infrastructure, not choices made by the operator. In the ZRB model, the canal-loss representations are calibrated once (see Section 7) and then held fixed across scenarios. Changing an allocation policy mainly changes **where** losses occur, not the fact that transport losses are built into the infrastructure.

For readability, some coursebook network figures suppress the reach nodes. The canal-loss results, however, are driven by those transport segments.

8.4 Event System and Traces

TaqSim is **event-sourced**: all system state is derived from an immutable log of events rather than from mutable state variables. Every time water is generated, received, stored, released, lost, consumed, distributed, or output, a corresponding event dataclass is recorded. This architecture enables full post-hoc reconstruction of any system state at any timestep, comparison across scenarios, and custom metric construction—all without re-running the simulation.

8.4.1 Common Event Families

Event family	Example events	Teaching meaning
Inflow and routing	<code>WaterGenerated</code> , <code>WaterReceived</code> , <code>WaterDistributed</code> , <code>WaterOutput</code>	How water enters the system and moves from one component to another

Event family	Example events	Teaching meaning
Storage and release	WaterStored, WaterReleased, WaterSpilled	How reservoirs and transport limits buffer, release, or reject water
Use and service	WaterConsumed, DeficitRecorded	How much demand is actually served and how much remains unmet
Losses	WaterLost	How water leaves through seepage, evaporation, inefficiency, overflow, or similar processes

The exact mix of events varies by node type, but the teaching idea is stable: every important water movement leaves an auditable trace in the event log.

8.4.2 Querying Events

Events can be queried in several ways:

- `node.events_of_type(WaterLost)` — all loss events across all timesteps
- `node.events_at(t)` — all events at a specific timestep
- `node.trace(WaterReleased, "amount")` — extract a time series of release amounts

8.4.3 Traces

A **Trace** is an immutable time-indexed series (`dict[int, float]`) that provides a rich interface for working with event-derived data:

Construction:

- `node.trace(EventType, field)` — extract from event log
- `Trace.from_events(events, field)` — from a list of events
- `Trace.from_dict({0: 100.0, 1: 150.0})` — from raw data
- `Trace.constant(50.0, range(TIMESTEPS))` — uniform value over timesteps

Arithmetic: Traces support element-wise operations with both other traces and scalars: `trace_a + trace_b`, `trace * 0.5`, `trace_a - trace_b`, `trace / scalar`. Binary operations use intersection semantics—only timesteps present in both traces appear in the result.

Transformation and aggregation:

Method	Description
<code>trace.map(fn)</code>	Apply a function to each value

Method	Description
<code>trace.filter(predicate)</code>	Keep only timesteps matching a condition
<code>trace.cumsum(initial)</code>	Running cumulative sum
<code>trace.sum()</code>	Total across all timesteps
<code>trace.mean()</code>	Average value
<code>trace.max()</code> , <code>trace.min()</code>	Extremes
<code>trace.get(t, default)</code>	Value at timestep <code>t</code> with default

In the ZRB coursebook, `compute_water_balance()` in `analysis.py` uses event queries to reconstruct the complete mass balance from the event log. The `trace()` method powers the time-series visualisations throughout the coursebook—reservoir storage trajectories, daily deficit patterns, and cumulative loss curves are all derived from Trace objects.

💡 Trace Arithmetic Uses Intersection Semantics

When combining two traces with `+`, `-`, `*`, or `/`, only **common timesteps** appear in the result. If one trace covers the full simulation and another covers only a shorter window, the result is limited to the overlap. Use `.get(t, 0.0)` when you need missing-timestep defaults, or `.reindex(timesteps, fill_value)` to extend a trace to a specific set of timesteps.

8.5 Strategies: Operational Policies vs Physical Models

TaqSim makes a fundamental distinction between **operational policies**—decisions about how to operate the system—and **physical models**—descriptions of how the system behaves. This distinction determines what the optimiser can and cannot change.

Category	Typical object	Optimisable	Purpose	ZRB examples
Operational policy	Release policy	yes	Decide how reservoirs release water over time	<code>ZRBReleaseRule</code> (SLOP)
Operational policy	Split policy	yes	Decide how splitters divide flow among branches	<code>MonthlyDistribution</code>
Physical model	Storage-loss model	no	Represent evaporation or seepage from stored water	Reservoir evaporation / seepage losses

Category	Typical object	Optimisable	Purpose	ZRB examples
Physical model	Reach routing or loss model	no	Represent lag, capacity, seepage, and evaporation along transport segments	Lag, CanalLossRule, EvaporationLossRule

8.5.1 The Strategy Base Class

All operational policies inherit from `Strategy`, which provides the machinery that makes optimisation possible. For teaching purposes, the important ideas are:

Strategy concept	Purpose
<code>__params__</code>	Names the tunable values on the strategy
<code>__bounds__</code>	Limits the permissible search range for those values
<code>__constraints__</code>	Enforces relationships such as ratios summing to 1
<code>__time_varying__</code>	Marks parameter blocks that vary across the seasonal cycle or across timesteps
<code>__cyclical__</code>	Marks time-varying parameters that repeat on a cycle

In other words, a strategy does not just tell the model how to operate; it also tells the optimiser **which parts are allowed to move** and **what counts as a valid configuration**.

8.5.2 Discovery and Validation

The optimiser-facing tooling walks the water system and collects only the **operational strategies** attached to nodes. Physical process models—such as reservoir loss models or reach loss models—are intentionally excluded, because they represent infrastructure behavior rather than operator choices.

Validation then ensures that candidate parameter sets remain meaningful: values must stay within bounds, and relationships such as ordered storage thresholds or split ratios summing to one must still hold.

8.5.3 Constraints

Constraints express relationships between parameters that must hold after genetic algorithm operations (crossover, mutation). Two especially important teaching examples are:

- `SumToOne(params)` — the named parameters must sum to a target value (default 1.0). Essential for split ratios: if a splitter has three downstream branches, their allocation ratios must sum to 1.0.
- `Ordered(low, high)` — one parameter must be less than or equal to another. Useful for SLOP zone boundaries where $\text{dead storage} < \text{buffer level} < \text{conservation level} < \text{capacity}$.

In the current ZRB model, `MonthlyDistribution` stores its ratio schedule directly on the strategy object, and the network builder supplies the repeating seasonal pattern used by the daily simulation. By contrast, `ZRBReleaseRule` still uses true monthly-cyclical tuples for its seasonal release parameters. This is a helpful reminder that “time-varying” can be implemented in more than one way, even within the same model family.

8.6 Parameter Exposure for Optimisation

Genetic algorithms operate on flat vectors of floating-point numbers, but TaqSim’s parameters are distributed across nested structures such as `system.nodes["dam"].release_policy.vr`. The **parameter exposure** system bridges this gap by providing a standardised interface to discover, flatten, and reconstruct parameters.

8.6.1 ParamSpec

Each tunable parameter is represented by a `ParamSpec(path, value)` where `path` is a dot-separated address (e.g., `"HW_Ravatkhoza.split_policy.ratio_HW_AkKaraDarya"`) and `value` is the current scalar value.

8.6.2 The Vectorisation API

Method	Returns	Purpose
<code>param_schema()</code>	<code>list[ParamSpec]</code>	Discover all tunable parameters with their paths and current values
<code>to_vector()</code>	<code>list[float]</code>	Flatten parameters to an optimiser-friendly vector
<code>with_vector(vec)</code>	<code>WaterSystem</code>	Create a new system with parameters from the vector (immutable)
<code>param_bounds()</code>	<code>dict[str, tuple]</code>	Bounds per parameter path

Method	Returns	Purpose
<code>bounds_vector()</code>	<code>list[tuple]</code>	Bounds matching <code>to_vector()</code> order
<code>constraint_specs()</code>	<code>list[ConstraintSpec]</code>	Resolved constraints for the repair function
<code>reset()</code>	<code>None</code>	Clear events and reset state for fresh simulation

A typical optimisation setup:

```
# Discover the parameter space
schema = system.param_schema()
for spec in schema:
    print(f"{spec.path} = {spec.value}")

# Flatten to vector
vector = system.to_vector()          # e.g., [0.6, 0.2, 0.2, ...]
bounds = system.bounds_vector()      # e.g., [(0.0, 1.0), (0.0, 1.0), ...]

# Reconstruct from a candidate vector (original unchanged)
candidate = system.with_vector(new_vector)
candidate.simulate(TIMESTEPS)
```

8.6.3 Time-Varying Parameter Expansion

When a strategy parameter is time-varying—for example a seasonal block of release coefficients or an indexed allocation schedule—`param_schema()` expands it into indexed entries. The vector flattens these into consecutive elements, and `with_vector()` reconstructs the structured parameter block. Bounds are replicated for each index.

8.6.4 Constraint Repair

After crossover and mutation, a candidate vector may violate bounds or constraints (e.g., split ratios no longer sum to 1.0). `make_repair(system)` creates a repair function that:

1. Clips each value to its parameter bounds
2. Applies constraint repairs (e.g., renormalises SumToOne groups)
3. Returns a valid vector

This repair function is applied to every candidate before evaluation, ensuring the optimiser only ever simulates physically meaningful configurations.

8.7 Objectives: Evaluating System Performance

An **Objective** wraps a function that evaluates one dimension of system performance after simulation:

```
Objective(  
    name="agricultural_deficit",  
    direction="minimize",  
    evaluate=lambda system: sum_deficit(system, "Dargom"),  
    priority=1,  
)
```

The `direction` specifies whether the optimiser should minimise or maximise the value. The `evaluate` function takes a simulated `WaterSystem` and returns a single float.

8.7.1 Built-In Objectives

TaqSim provides a fluent registry API for common objectives:

- `minimize.spill(node_id)` — total volume spilled at a Storage or PassThrough node
- `minimize.deficit(node_id)` — total unmet demand at a Demand node

These cover the most common water system goals: reducing waste (spill) and improving service (deficit). Custom objectives can evaluate any function of the event log—for example, hydropower generation derived from flow traces, environmental flow compliance, or reservoir storage reliability.

8.7.2 The `lift` Decorator

The `lift` decorator transforms scalar functions to work on `Trace` objects, enabling physics-based objective construction. For example, a hydropower objective might lift a power function ($P = gQh$) to operate on flow and head traces, then aggregate the result.

8.7.3 ZRB Application

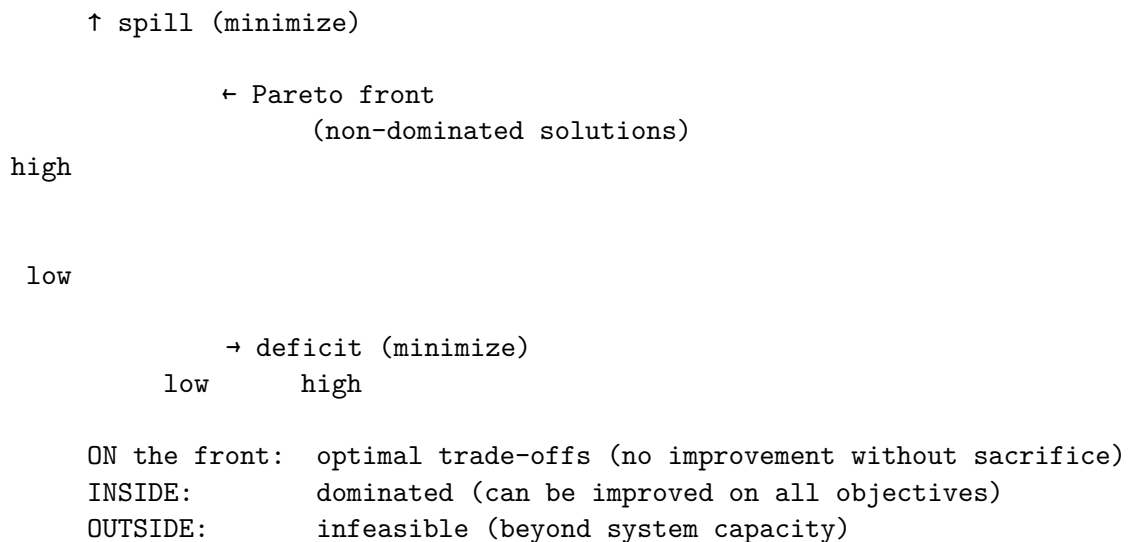
The companion optimisation coursebook uses `minimize.deficit()` for each irrigation district and `minimize.spill()` for the two reservoirs. The optimiser simultaneously searches for splitting ratios and release parameters that best balance these competing objectives.

8.8 Multi-Objective Optimisation

The strategies explored in this coursebook—uniform, static demand-proportional, and monthly-varying splitting—are heuristics. They use domain knowledge to construct reasonable operating rules, but they cannot navigate the trade-offs between competing objectives. What if improving Dargom’s delivery worsens Narpay’s? What if reducing agricultural deficit increases reservoir spillage? These are inherently **multi-objective** problems.

8.8.1 Dominance and the Pareto Front

Solution A **dominates** solution B if A is at least as good on every objective and strictly better on at least one. A solution that is not dominated by any other is called **Pareto optimal** (or non-dominated). The set of all Pareto-optimal solutions forms the **Pareto front**—the boundary of achievable performance:



The shape of the front reveals the cost of trade-offs. **Steep sections** mean large spill reductions for small deficit increases; **flat sections** mean large deficit reductions for small spill increases. **Knee points**—where the front bends sharply—often represent attractive compromise solutions.

8.8.2 NSGA-II via ctrl-freak

The optimisation engine is **ctrl-freak** (Lazaro and Siegfried 2025), a pure-numpy genetic algorithm framework that implements **NSGA-II** (Non-dominated Sorting Genetic Algorithm II) with pluggable selection and survival strategies. ctrl-freak is algorithm-only: it knows nothing about water systems. TaqSim’s parameter exposure system (Section 8.6) bridges the

gap by flattening system parameters into the flat numeric vectors that genetic algorithms operate on and reconstructing configured water systems from candidate vectors.

The NSGA-II algorithm maintains a population of candidate solutions and evolves them over generations:

1. **Initialise** a random population within parameter bounds
2. **Evaluate** all objectives for each candidate (simulate → compute objectives)
3. **Repeat** for N generations:
 - a. Select parents via crowded tournament (prefer lower Pareto rank, then higher crowding distance for diversity)
 - b. Create offspring via crossover (SBX) and mutation (polynomial); apply repair to enforce bounds and constraints
 - c. Combine parents and offspring; rank by non-dominated sorting
 - d. Select survivors to fill the next generation
4. **Return** an `NSGA2Result` containing the final population, Pareto ranks, and crowding distances

8.8.3 The `ctrl_freak.nsga2()` API

The optimisation loop connects `ctrl-freak`'s algorithm to `TaqSim`'s simulation via four user-supplied functions:

```
from ctrl_freak import nsga2

def init(rng):
    """Initialise one candidate: random parameter vector within bounds."""
    return rng.uniform(lower_bounds, upper_bounds)

def evaluate(x):
    """Simulate one candidate and return objective values."""
    candidate = system.with_vector(x)    # reconstruct WaterSystem
    candidate.simulate(TIMESTEPS)         # run the full daily simulation
    return np.array([obj.evaluate(candidate) for obj in objectives])

def crossover(p1, p2):
    """Recombine two parent vectors (e.g., SBX crossover)."""
    return sbx_crossover(p1, p2)

def mutate(x):
    """Perturb a candidate vector, then repair constraints."""
    return repair(polynomial_mutation(x))

result = nsga2(
    init=init,
```

```

    evaluate=evaluate,
    crossover=crossover,
    mutate=mutate,
    pop_size=100,
    n_generations=200,
    seed=42,
    n_workers=4,
)

```

The result is an `NSGA2Result` (frozen dataclass) containing:

- `result.population` — a `Population` with decision vectors (`.x`, shape $N \times d$) and objective values (`.objectives`, shape $N \times m$)
- `result.rank` — Pareto rank per individual (rank 0 = non-dominated front)
- `result.crowding_distance` — diversity metric per individual
- `result.pareto_front` — convenience property that extracts the rank-0 subset as a new `Population`

To inspect a specific Pareto-optimal solution:

```

front = result.pareto_front          # Population of non-dominated solutions
best_x = front.x[0]                  # decision vector of first front member
system_star = system.with_vector(best_x)
system_star.simulate(TIMESTEPS)
wb_star = compute_water_balance(system_star)

```

The `repair` function created by `make_repair(system)` ensures that every candidate evaluated by the algorithm is physically valid: parameter bounds are enforced, and structural constraints (e.g., split ratios summing to 1.0) are restored after crossover and mutation (Section 8.6).

! No Single Best Solution

The Pareto front does not contain a single best solution. Every point on the front is equally “optimal” in the Pareto sense—improving one objective necessarily worsens another. Selecting a final solution requires **human judgment** about priorities: risk-averse operators might prefer low-deficit solutions; infrastructure planners might accept higher deficits to reduce reservoir stress. The Pareto front provides options; stakeholders provide values.

9 Appendix C: zarafshan_taqsim Module Reference

The `zarafshan_taqsim` package provides a complete implementation of the ZRB water system with six modules. Each module is introduced here for reference; the coursebook sections where each module is first used are noted in parentheses.

- **data.py** loads the historical source datasets from the `data/ZRB_baseline/` directory—river inflow, irrigation demand for six districts, precipitation, evaporation, minimum environmental flows, canal properties, and reservoir height-volume curves. In the current coursebook workflow, these source series are translated into daily model inputs for the 2017–2022 simulation period. Primary hydrological data (river inflow, irrigation demand, minimum flows) are derived from Ragettli et al. (2025), with network topology from Giovanoli (2025b) and canal properties estimated from GIS and literature sources. (First used: Section 2)
- **strategies.py** implements operating rules specific to the ZRB: a Storage Level Operating Policy (SLOP) for reservoirs (`ZRBReleaseRule`), seasonally varying allocation ratios for splitters (`MonthlyDistribution`), and canal loss models (`CanalLossRule`, `EvaporationLossRule`). The `MonthlyDistribution` strategy is the key mechanism that enables the three-strategy comparison: it stores the ratio schedule used by each splitter—365 repeated daily entries for the static baseline, or a repeating seasonal pattern derived from monthly demand for the monthly-varying baseline. (First used: Section 4)
- **network.py** assembles the full daily ZRB water system via `create_zrb_system()`, provides `visualize_network()` for directed-graph visualisation, and implements the demand-proportional splitting functions `apply_static_demand_splitting()` and `apply_demand_proportional_splitting()` that compute allocation ratios from downstream demand patterns. (First used: Section 2.1)
- **analysis.py** provides the `WaterBalance` dataclass and `compute_water_balance()` function—the central post-simulation analysis tool. It reconstructs a complete mass balance from TaqSim’s event log, tracking inflow, consumption, losses (seepage, evaporation, operational, overflow), storage changes, and closure error. (First used: Section 4.1)
- **visualization.py** contains reporting and dashboard functions used throughout the coursebook: `print_annual_water_balance()` for formatted water balance tables, `plot_system_dashboard()` for four-panel Plotly dashboards, and comparison utilities for side-by-side strategy evaluation. (First used: Section 4.1)
- **objectives.py** defines objective functions that evaluate system performance after simulation, designed to be called from within `ctrl-freak`’s `evaluate` callback (Section 8.8). Key functions include `total_agricultural_deficit()`, `equity_deficit()`,

`reservoir_spillage()`, `total_spillage()`, `sink_min_flow_deficit()`, `passthrough_min_flow_deficit()` and `release_variability()`. Each takes a simulated `WaterSystem` and returns a single float suitable for minimisation. (First used: companion optimisation coursebook)

10 Appendix D: Glossary

Term	Definition
Closure error	Residual in the water balance equation (Inflow – Consumed – Sink outflow – Losses – Δ Storage); a value near zero confirms mass conservation.
Constraint	A relationship between strategy parameters that must hold after genetic operations (e.g., allocation ratios must sum to 1.0). Built-in types: <code>SumToOne</code> , <code>Ordered</code> .
Dead storage	Water stored below a reservoir’s intake structure that cannot be released; set at 10% of capacity in this model.
Effective delivery	Water delivered to a district up to (but not exceeding) its demand in each timestep. Delivery beyond demand is counted as over-delivery, not effective delivery.
Effective efficiency	Effective delivery / total demand. The fairest single metric for comparing allocation strategies because it does not reward over-delivery.
Reach loss model	A physical model applied to transport segments to represent seepage, evaporation, or related conveyance losses. It is not an operational policy and is therefore not an optimisation target.
Event-sourced	An architecture where system state is reconstructed from a log of immutable events (e.g., <code>WaterReceived</code> , <code>WaterLost</code>) rather than tracked via mutable state variables.
HydroWorks	Diversion structures (modelled as Splitter nodes) that divide river flow among competing downstream branches.
Interbasin transfer	Water conveyed from one river basin to another (e.g., from the Zarafshan to Jizzakh province in the Syrdarya basin).
NSGA-II	Non-dominated Sorting Genetic Algorithm II—a multi-objective evolutionary algorithm used in the companion optimisation coursebook to find Pareto-optimal solutions.
Objective	A named function evaluating system performance on one dimension (e.g., total deficit), with a direction (minimize or maximize). See Section 8.7.
Over-delivery	Water delivered to a district beyond its requirement in a given timestep. Provides no additional benefit at that district but may flow through to downstream nodes.

Term	Definition
Parameter exposure	The mechanism by which TaqSim exposes strategy parameters as a flat vector of floats for genetic algorithm optimisation. See Section 8.6.
Pareto front	The set of solutions where no objective can be improved without worsening at least one other objective. Reveals the trade-off surface between competing goals.
Pass-through	Behaviour of demand nodes where water delivered in excess of requirement continues downstream to subsequent nodes rather than being lost.
SLOP	Storage Level Operating Policy—a rule that determines reservoir release rates based on four storage zones: dead, buffer, conservation, and flood control.
Strategy	A base class for operational policies (such as release and split policies) whose parameters are tunable by optimisation. Declares metadata such as <code>__params__</code> , <code>__bounds__</code> , and <code>__constraints__</code> . See Section 8.5.
System efficiency	Total delivery at district gates / total inflow. Can be misleading because it counts all delivery, including water delivered where it is not needed.
Topological sort	An ordering of directed-graph nodes such that every node is processed after all its upstream predecessors. Used by TaqSim to determine simulation execution order.
Trace	An immutable time-indexed series (timestep $\rightarrow$ value) extracted from event logs, supporting arithmetic, transformation, and aggregation operations. See Section 8.4.
Waste fraction	Over-delivery / total inflow. Measures the fraction of system inflow delivered beyond what any district requires.
Water balance	A mass-conservation accounting that tracks all water from inflow through to consumption, losses, storage changes, and terminal outflow.

References

- CAWater-Info. 2024a. “Reservoirs in Uzbekistan.” https://cawater-info.net/bk/1-1-1-1-3-uz_e.htm.
- CAWater-Info. 2024b. “Water Resources of Zeravshan River Basin.” https://www.cawater-info.net/zeravshan/water_e.htm.
- FAO. 1989. *Irrigation Water Management: Irrigation Scheduling*. Training Manual No. 4 (T7202/E). Food; Agriculture Organization of the United Nations. <https://www.fao.org/3/T7202E/T7202E00.htm>.
- Giovanoli, Gianna. 2025a. *Cooling Water Requirements of the Navoi Thermal Power Plant*. Personal communication with local expert, Navoi Province, Uzbekistan.
- Giovanoli, Gianna. 2025b. “Optimising Water Allocation in the Zarafshan River Basin Under Climate Change.” MSc Thesis, ETH Zurich.
- Global Energy Monitor. 2024. “Navoi Power Station.” https://www.gem.wiki/Navoi_power_station.
- Groll, Michael, Christian Opp, Rashid Kulmatov, Malika Ikramova, and Inom Normatov. 2015. “Water Quality, Potential Conflicts and Solutions—an Upstream–Downstream Analysis of the Transnational Zarafshan River (Tajikistan, Uzbekistan).” *Environmental Earth Sciences* 73 (2): 743–63. <https://doi.org/10.1007/s12665-013-2988-5>.
- Karimov, B. K., B. G. Kamilov, and D. Keyser. 1995. *Fisheries in the Zarafshan River Basin (Uzbekistan)*. FAO Fisheries Circular No. 894. Food; Agriculture Organization of the United Nations. <https://www.fao.org/4/v9529e/v9529e06.htm>.
- Khodjiyev, A., and S. Atadjanova. 2022. “The Current State of Irrigation Networks and Their Use in the Water Sector of the Republic of Uzbekistan.” *E3S Web of Conferences* 365: 01011. <https://doi.org/10.1051/e3sconf/202336501011>.
- Kulmatov, Rashid, Christian Opp, Michael Groll, and Dilafruz Kulmatova. 2013. “Assessment of Water Quality of the Trans-Boundary Zarafshan River in the Territory of Uzbekistan.” *Journal of Water Resource and Protection* 5 (1): 17–30. <https://doi.org/10.4236/jwarp.2013.51003>.
- Lazaro, Nicolas, and Tobias Siegfried. 2024. *TaqSim: An Event-Sourced Water System Simu-*

- lation Framework*. Hydrosolutions GmbH, released. <https://github.com/hydrosolutions/taqsim>.
- Lazaro, Nicolas, and Tobias Siegfried. 2025. *ctrl-freak: An Extensible Genetic Algorithm Framework*. Hydrosolutions GmbH, released. <https://github.com/hydrosolutions/ctrl-freak>.
- Lillis, Joanna. 2020. “Uzbekistan: Where the Amu Darya Goes to Die.” *Eurasianet*. <https://eurasianet.org/uzbekistan-where-the-amu-darya-goes-to-die>.
- Ragettli, Silvan, Adrian Kreiner, Andrey Yakovlev, et al. 2025. “Unraveling Agricultural Water Use in Three Central Asian Irrigation Oases Using Remote Sensing.” *Journal of Hydrology: Regional Studies* 58: 102185. <https://doi.org/10.1016/j.ejrh.2025.102185>.
- Rakhmatullaev, Shavkat, Frédéric Huneau, Hélène Celle-Jeanton, Philippe Le Coustumer, Mikael Motelica-Heino, and Masharif Bakiev. 2011. “Geostatistical Approach for the Assessment of the Water Reservoir Capacity in Arid Regions: A Case Study of the Akdarya Reservoir, Uzbekistan.” *Environmental Earth Sciences* 63 (3): 447–60. <https://doi.org/10.1007/s12665-010-0711-3>.
- Shibuo, Y., J. Jarsjö, and G. Destouni. 2007. *Hydrological Responses to Climate Change and Irrigation in the Aral Sea Drainage Basin*. Stockholm University, Department of Physical Geography.
- Shultz, Viktor L. 1965. *Rivers of Middle Asia*. 2nd ed. Gidrometeoizdat.
- USAID. n.d. *Irrigation Map of the Zarafshan Valley* (). United States Agency for International Development, Central Asia Natural Resources Management Project (NRMP).
- Zaman, M., G. Fang, M. Saifullah, and A. Raza. 2023. “Characterization and Control of Irrigation Canal Seepage Losses: A Review.” *Agricultural Water Management* 287: 108438. <https://doi.org/10.1016/j.agwat.2023.108438>.